A Survey of Agentic Reinforcement Learning for Large Language Models: Foundations, Paradigms, and Challenges

USTC-AGI Research Group
University of Science and Technology of China
mycheng@ustc.edu.cn

April 23, 2026

Abstract

Agentic reinforcement learning (agentic RL) is best understood as the transition from single-turn preference or verifier optimization to multi-turn policy learning in environment-coupled settings. Unlike RLHF and RLVR, which remain bandit formulations over complete responses, agentic RL places the model in a stateful world where tool use, intermediate observations, and persistent world changes make learning sequential and partially observed. This survey studies that transition while treating RLHF and RLVR as essential precursors: they provide the conceptual and algorithmic background needed to understand what transfers to multi-turn settings and what breaks. Our organizing lens is therefore *interaction structure*, from single-turn feedback to multi-turn interaction, and our main analytic focus is the end-to-end training decomposition that this shift induces across algorithms, trajectory/data design, and training infrastructure. On this basis, we formalize agentic RL as a POMDP, develop taxonomies for problem structure and credit assignment, compare trajectory representation paradigms in terms of serialization fidelity and policy-loss scope, and synthesize representative empirical patterns and infrastructure design choices across modern agentic RL systems.

Contents

1 Introduction **4**
1.1 From Single-Turn Feedback to Multi-Turn Interaction . . . 4
1.2 Scope of This Survey . . . 5
1.3 Relationship to Existing Surveys . . . 5
1.4 Survey Methodology . . . 6
1.5 Contributions . . . 8
1.6 Organization . . . 9

2 Background: From RLHF to RLVR **10**
2.1 The Alignment Problem and the Role of RL . . . 10
2.2 Generation 1: Reinforcement Learning from Human Feedback . . . 10
2.2.1 The Three-Stage Pipeline . . . 10
2.2.2 PPO in the Language Model Context . . . 11
2.2.3 Extensions: Constitutional AI and RLAIF . . . 11
2.2.4 Offline Preference Optimization as a Side Branch . . . 12
2.2.5 Limitations of RLHF . . . 12
2.3 Generation 2: Reinforcement Learning with Verifiable Rewards . . . 12

2.3.1 The Verifiable Reward Hypothesis . . . 12
2.3.2 GRPO: Group Relative Policy Optimization . . . 13
2.3.3 Reasoning as Part of the Policy . . . 13
2.3.4 Reward Design for Verifiable Tasks . . . 13
2.3.5 Limitations of RLVR . . . 14

3 Agentic RL: Concepts and Formal Definitions 14
3.1 Formal Problem Formulation . . . 14
3.2 Taxonomy of Agentic RL Problems . . . 17
3.3 The ReAct Interaction Protocol . . . 18
3.4 Relationship to Classical RL . . . 18

4 Agentic RL: Unique Challenges 19
4.1 Credit Assignment over Long Horizons . . . 19
4.2 Long Context and Computational Cost . . . 19
4.3 Rollout Cost Inversion . . . 20
4.4 Actions with Persistent Consequences . . . 20
4.5 Reward Hacking and Proxy Exploitation . . . 20
4.6 Policy Collapse in Multi-Turn RL . . . 20
4.7 Partial Observability and Environmental Non-Determinism . . . 21
4.8 Training Distribution Shift . . . 21

5 Trajectory Representation and Sampling in Agentic RL 21
5.1 Formal Definition of Agent Trajectories . . . 21
5.2 Trajectory Representation Paradigms . . . 22
5.3 Rollout Collection Pipelines . . . 24
5.4 Tool Call Serialization and Token-Alignment Fidelity . . . 24
5.5 Reward Attribution and Policy-Loss Scope . . . 24
5.6 Trajectory Sampling Strategies . . . 24
5.7 Trajectory Quality Filtering and Negative Mining . . . 25
5.8 Historical Trajectories and Warm-Starting . . . 25
5.9 Trajectory Truncation and Compression for Long-Context Agents . . . 25

6 Algorithms for Agentic RL 26
6.1 A Credit-Locus View of Agentic RL Algorithms . . . 26
6.2 Trajectory-Level Credit Assignment . . . 27
6.3 Step-Level Credit Assignment . . . 28
6.4 Token-Level Credit and Loss Scope . . . 28
6.5 Structured Exploration and Multi-Agent Extensions . . . 29

7 Training Infrastructure for Agentic RL 30
7.1 Two Infrastructure Shifts . . . 30
7.2 From LLM Training/Inference to LLM RL Training/Inference . . . 30
7.3 Single-Turn LLM RL: Three Representative Abstractions . . . 30
7.4 From Single-Turn RL to Multi-Turn Agentic RL . . . 31
7.5 Training-Agent Disaggregation and Rollout-as-a-Service . . . 32
7.6 Supporting Systems: Sandboxes, Evaluation, and Reproducibility . . . 33
7.7 Framework Comparison . . . 33

8 Applications and Benchmarks **34**
8.1 Application Domain Overview . . . 34
8.2 Agentic RAG and Deep Research . . . 34
8.3 Web Navigation Agents . . . 34
8.4 Coding and Software Engineering Agents . . . 35
8.5 GUI and Computer-Use Agents . . . 35
8.6 Game-Like and Embodied Agents . . . 35
8.7 Structured Data and Analytical Agents . . . 36
8.8 AutoResearch and Data Science Agents . . . 36
8.9 Collaborative Multi-Agent Systems . . . 37
8.10 Open-Ended Assistant Agents . . . 37
8.11 Benchmark and Evaluation Landscape . . . 37

9 Open Challenges and Future Directions **41**
9.1 Learning Signals and Supervision . . . 41
9.2 Initialization and Sample Efficiency . . . 42
9.3 Evaluation and Training Systems . . . 43
9.4 Theory and Scaling . . . 43

10 Conclusion **46**

1 Introduction

1.1 From Single-Turn Feedback to Multi-Turn Interaction

Consider two training examples drawn from different moments in the evolution toward agentic RL. In the first, a human annotator reads two short paragraphs of text—two completions of a prompt asking for a restaurant recommendation—and marks one as better, in the style of the preference-comparison pipeline popularized by InstructGPT [Ouyang et al., 2022]. A reward model learns from thousands of such comparisons, and a language model is finetuned to produce outputs that score highly under this reward. The entire feedback loop takes place in the space of static text. The environment, in the technical sense of an agent interacting with a world, is notional at best: the language model generates a single response, and the episode ends.

In the second example, a language model is given a GitHub issue and tasked with resolving it, as in end-to-end agent training settings such as Agent-R1 [Cheng et al., 2025]. Over the course of the episode, the model reads files, writes code, runs tests, interprets error messages, refines its patch, and submits a pull request. Each tool call changes the state of the environment—a Docker container running a pinned version of the repository—and the final reward depends on whether the modified codebase passes the repository's test suite. The feedback loop is now genuinely interactive: the model's actions have consequences that persist across turns, and success requires not just generating plausible text but reasoning about how actions compose over time. This is qualitatively different from long-form but still single-response reasoning settings such as DeepSeekMath [Shao et al., 2024].

These two examples differ not just in scale but in structure. The first is a *bandit*: a single observation, a single action, a single reward. The second is a partially observed sequential decision problem with stochastic dynamics, long horizons, and sparse reward. The techniques and infrastructure do not transfer cleanly between them.

The organizing perspective of this survey is that *agentic RL* is best understood through *interaction structure*: a progression from *single-turn feedback* to *multi-turn interaction*. In this progression, RLHF and RLVR serve as important single-turn precursors, whereas agentic RL occupies the genuinely interactive regime: actions change future observations, and performance depends on how decisions compose over time. This structural shift, summarized in Table 1, motivates the algorithmic and systems questions emphasized throughout the survey.

Table 1: The three generations of RL for LLMs, characterized by the structure of the feedback environment.

Generation	**Environment**	**Interaction Type**	**Decision Structure**	**Core Bottleneck**
Gen 1: RLHF	Human preference annotations	Single-turn	Single-step bandit	Annotation throughput
Gen 2: RLVR	Verifiable reward function	Single-turn	Single-step bandit	Requires verifiable tasks
Gen 3: Agentic RL	Interactive world	Multi-turn	Multi-step POMDP	Environment interaction cost

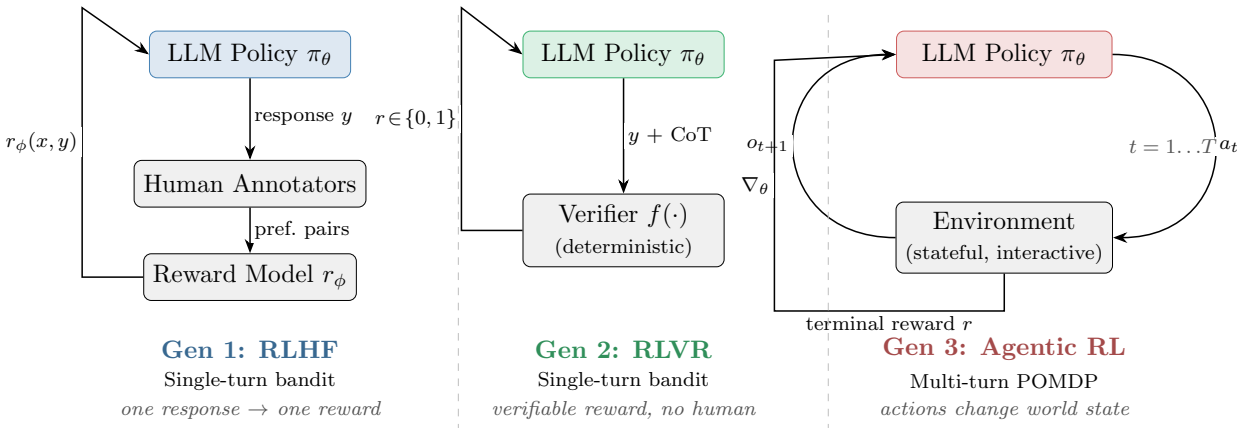

Figure 1: **The three generations of RL for LLMs, distinguished by feedback-loop structure.** *Gen 1* (RLHF) trains a reward model r_ϕ on human preference annotations and optimises the LLM policy against it in a single-turn bandit setting. *Gen 2* (RLVR) replaces the learned reward model with a deterministic verifier—still single-turn, but with higher-fidelity and scalable reward signals that eliminate the annotation bottleneck. *Gen 3* (Agentic RL) replaces the static evaluator with a stateful interactive environment: the policy issues actions a_t (tool calls, code, messages), receives observations o_{t+1}, and completes T turns before a terminal reward drives a policy gradient update. The shift from a single-turn bandit to a multi-turn POMDP is not a change in scale but a categorical expansion of the learning problem, requiring new algorithms, trajectory representations, and training infrastructure.

1.2 Scope of This Survey

This survey covers reinforcement learning *applied to* LLMs—methods that use RL to improve LLM behavior—rather than LLMs *applied to* RL problems. Within that space, it does not aim to cover all RL-for-LLM regimes at equal depth, nor to provide a broad survey of LLM agents as a whole. Its primary focus is the *agentic RL* regime: the point at which training moves from single-turn feedback over complete responses to multi-turn learning in stateful environments. RLHF and RLVR are therefore treated as essential preliminaries rather than co-equal endpoints. They establish the conceptual, algorithmic, and empirical background needed to explain what transfers to the agentic setting and what breaks once the problem becomes sequential, partially observed, and environment-coupled. The main body then centers on the three coupled dimensions that become first-order in that regime: algorithms, trajectory/data design, and training infrastructure.

1.3 Relationship to Existing Surveys

Early surveys such as Xi et al. [2023] and Wang et al. [2024b] provide broad overviews of LLM-based agents, but treat RL as one component in a larger agent stack. Casper et al. [2023] offers a detailed critique of RLHF, yet predates the recent wave of RLVR and agentic RL systems. Representative recent surveys include a dedicated landscape view of agentic RL [Zhang et al., 2026], a lifecycle-oriented survey of RL for LLMs [Liu et al., 2025a], an algorithm-centric technical survey [Srivastava and Aggarwal, 2025], a search-focused vertical survey [Lin et al., 2025], a model-native agentic AI survey [Sang et al., 2025], a broad agentic LLM survey [Plaat et al., 2025], and a survey of multi-turn LLM interaction more broadly [Li et al., 2026]. Among these, the closest published survey to our topic is the TMLR survey by Zhang et al. [2026], which organizes the field primarily around agentic capabilities, application domains, and an accompanying resource landscape of environments,

benchmarks, and frameworks. The present survey is intended to be complementary rather than redundant. Its distinguishing emphasis is twofold: first, it treats the shift from single-step bandits to multi-step POMDPs as the defining structural transition into agentic RL; second, it uses that transition to organize the practical training decomposition of the regime across algorithms, trajectory representation/data design, and infrastructure.

Table 2: Comparison with representative recent surveys related to RL for LLMs. Coverage labels are qualitative: *High* = central focus, *Med.* = recurring but not primary, *Low* = brief or selective discussion, and *None* = no dedicated coverage.

Survey	**Primary Lens**	**RLHF / RLVR**	**Agentic RL**	**Trajectory / Data**	**Infrastructure**	**Apps / Benchmarks**
Agentic RL Landscape (2026)	Agentic capability, task, and resource landscape	Med.	High	Low	Low	High
LLM Lifecycle (2025)	RL across pre-training, alignment, and reasoning	High	Low	None	Low	Med.
RL Techniques (2025)	RL algorithms for alignment and reasoning	High	Low	None	None	Low
Agentic Search (2025)	Search as an agentic RL vertical	Med.	Med.	None	None	High
Model-native Agentic AI (2025)	Pipeline-to-model-native transition	Med.	Med.	Low	None	Med.
Agentic LLMs (2025)	Broad reason–act–interact framing	Low	Low	None	None	Med.
Multi-Turn LLMs (2026)	Multi-turn interaction beyond RL alone	Low	Low	Low	None	Med.
This survey	Interaction structure and end-to-end training pipeline	High	High	High	High	High

Across this literature, trajectory handling and training infrastructure are usually secondary to broader discussions of capabilities, applications, or alignment algorithms. Yet in agentic RL, trajectories are not just examples or benchmark traces: how they are *represented*, filtered, segmented, truncated, and reused directly affects which learning algorithms are feasible. The same is true for *infrastructure*: asynchronous rollout, sandbox execution, trainer–worker decoupling, and data freshness are not merely framework features, but part of the practical design space of agentic RL.

Against this background, the present survey emphasizes three aspects. First, it places *agentic RL* at the center while using RLHF and RLVR to clarify the transition from single-turn to multi-turn learning. Second, it treats *algorithms*, *trajectory/data design*, and *training infrastructure* as three coupled parts of one training pipeline. Third, it approaches the agentic regime not only as a collection of applications or capabilities, but also as a systems-and-learning co-design problem.

1.4 Survey Methodology

This survey is a structured narrative review rather than a formal systematic review or meta-analysis. Our primary goal is not exhaustive enumeration of every paper touching LLM agents or RL, but a technically coherent map of the subliterature most relevant to *training* LLM agents with reinforcement learning. Accordingly, we prioritize original papers that introduce, analyze, or operationalize methods used in practice.

Search Scope and Temporal Cutoff. The literature considered spans the modern RL-for-LLM period from 2019 through March 31, 2026. Our core search space includes foundational RLHF

and RLVR papers, agentic RL algorithm papers, infrastructure and systems papers, benchmark papers, and closely related surveys used for positioning. We prioritize primary sources—original model, algorithm, benchmark, and systems papers—over secondary commentary. This temporal cutoff therefore includes late-Q1 2026 papers that sharpen emerging themes in the field, such as multi-agent rollout–training co-design in FlexMARL [Jiang et al., 2026] and turn-level reward shaping for long-context memory decisions in TAMTRL [Wang et al., 2026].

Inclusion Criteria. We include work if it makes a substantive contribution to at least one of the three central axes of this survey: *algorithms*, *trajectories and data*, or *training infrastructure*. We also include benchmark and application papers when they clarify the structure of the environment, reward, or evaluation protocol used by agentic RL systems. Foundational pre-agentic work (e.g., RLHF, DPO, RLVR) is included when it is necessary to explain what transfers to the multi-turn setting and what breaks.

Exclusion and Boundary Decisions. We generally exclude purely prompt-engineering papers, static agent workflow papers without a reinforcement learning component, and broad LLM-agent surveys that do not materially inform the training pipeline. We nevertheless retain a small number of non-RL baselines—such as ReAct and SWE-agent—when they are historically load-bearing for understanding the execution protocol, action interface, or benchmark scaffold against which later RL systems are evaluated.

Why These Representative Works. Within each topic, representative papers are selected to cover one or more of the following roles: an early canonical formulation, an influential algorithmic refinement, a systems design that changes what is practically trainable, an open-source framework that shapes community practice, or a benchmark/application paper that exposes a distinctive agentic challenge. When several papers make highly overlapping points, we discuss the paper that most clearly exposes the relevant technical idea and mention the others selectively to avoid turning the survey into an unstructured catalogue.

Table 3 summarizes how these inclusion decisions map to the major paper categories that recur throughout the survey.

Table 3: Paper selection summary used in this survey.

Category	Included When	Primary Role in This Survey
Foundational RL for LLMs	The paper defines or materially changes the RLHF, DPO, or RLVR baseline that agentic methods inherit from	Establishes the pre-agentic baseline and clarifies which assumptions transfer to multi-turn settings
Agentic RL algorithms	The paper introduces a distinct training objective, credit assignment mechanism, exploration strategy, or replay/relabeling design for interactive agents	Forms the core algorithmic comparison in Sections 6 and 9
Systems and infrastructure	The paper changes rollout collection, trainer–worker decoupling, sandbox execution, or distributed scheduling in a way that affects what is trainable at scale	Grounds the infrastructure analysis in Section 7
Frameworks and toolchains	The framework is open-source or otherwise influential enough to shape community practice and experimental design	Illustrates reusable implementations and design patterns rather than new theory
Benchmarks and applications	The benchmark or domain paper exposes a distinctive environment, reward structure, or evaluation protocol relevant to agentic RL	Anchors the taxonomy and empirical discussion in concrete tasks and environments
Prompting / workflow baselines	The work lacks RL training but is historically load-bearing for the agent loop, interface, or benchmark scaffold	Serves as a non-RL baseline or protocol precursor, e.g., ReAct and SWE-agent

1.5 Contributions

This survey makes one central claim and develops it through six concrete contributions. The central claim is that agentic RL should be understood as the transition from single-turn feedback optimization to multi-turn environment-coupled policy learning, and that this transition reorganizes the practical training problem around three coupled dimensions: algorithms, trajectory/data design, and infrastructure. On that basis, the survey first gives a formal POMDP specification $(S, A, O, T, E, R, \gamma)$ for agentic RL with LLM policies, providing a shared vocabulary for comparing assumptions and optimization choices. Second, it develops a four-dimensional taxonomy of agentic RL problems across action space, time horizon, reward structure, and observability, and situates representative benchmarks such as HumanEval, SWE-bench, WebArena, and GAIA within that space. Third, it develops a five-level credit assignment taxonomy spanning token, segment/step, turn, trajectory/goal, and agent loci, together with the main estimation and normalization mechanisms that cut across them.

Fourth, it compares three trajectory representation paradigms—message-based, Token-In-Token-Out (TITO), and step-based—through the lenses of serialization fidelity, explicit token-level masking requirements, and credit assignment compatibility. Fifth, it synthesizes representative empirical patterns across selected agentic RL systems. Sixth, it treats training infrastructure—including Rollout-as-a-Service, asynchronous scheduling, sandbox execution, and trainer–worker decoupling—

as a first-class research dimension rather than an implementation afterthought.

1.6 Organization

The remainder of this survey is organized as follows. Section 2 reviews the first two generations of RL for LLMs, covering RLHF and RLVR in sufficient depth to motivate the transition to agentic settings. Section 3 formalizes the agentic RL problem, establishing notation and taxonomy used throughout. Section 4 characterizes the unique technical challenges that distinguish agentic RL from its predecessors. Sections 5, 6, and 7 constitute the survey's main body, covering trajectories and data, algorithms, and training infrastructure respectively. Section 8 surveys applications and benchmarks. Section 9 discusses open research questions. Section 10 concludes.

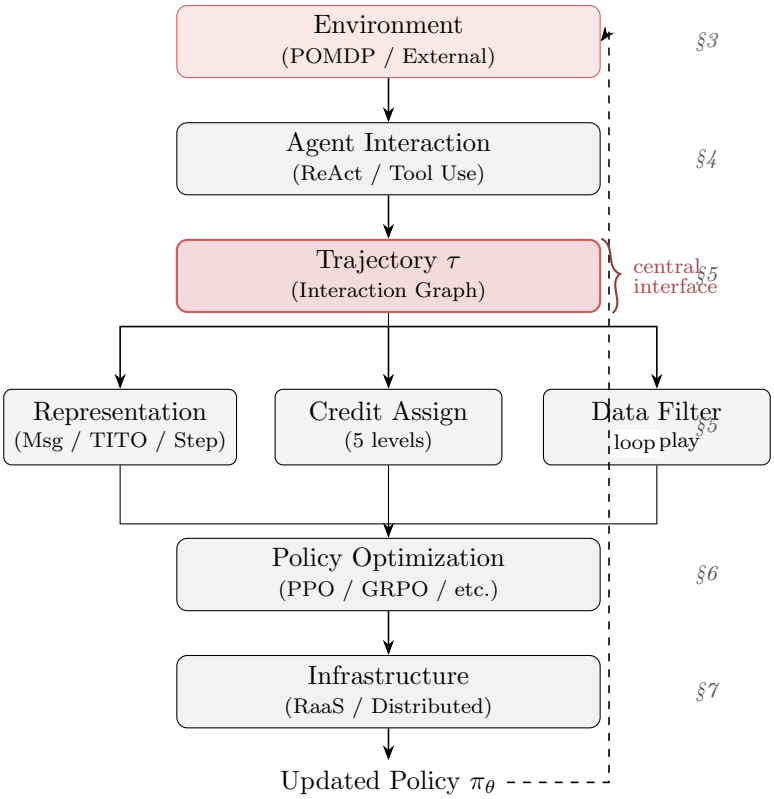

Figure 2: **Overview of the agentic RL training pipeline.** Unlike RLHF and RLVR, agentic RL is a **closed-loop system**: the policy interacts with a stateful environment, generates multi-turn trajectories, and is optimised based on delayed rewards—repeating continuously. **Trajectory τ is the central interface** connecting the environment, trajectory data design (representation, credit assignment, filtering), and policy optimisation; it is the primary artifact through which all other components communicate. Crucially, **algorithm choice is not the dominant factor** in agentic RL; system design (infrastructure, rollout organisation) and trajectory structure (representation paradigm, credit assignment granularity) are equally critical to training stability and final performance. Dashed sections annotate the survey chapters covering each component.

2 Background: From RLHF to RLVR

The development of RL for LLMs began with a concrete practical problem: large language models trained on internet text learned to predict text, but not to follow instructions or align with human values. This section traces the two pre-agentic generations developed to address that gap, establishing the conceptual vocabulary and technical baseline against which agentic RL will be measured. The organizing thread is the shift in supervision source and reward specification: RLHF learns from human preferences, whereas RLVR replaces those learned preferences with externally checkable rewards. Within that main arc, we briefly note offline preference-optimization alternatives such as DPO and its variants, because they reshape the RLHF family without changing the underlying single-turn problem class.

2.1 The Alignment Problem and the Role of RL

A language model trained by next-token prediction on a large corpus learns the statistical structure of human text, but human text includes misinformation, bias, harmful content, and countless examples of poor reasoning. More fundamentally, the objective of predicting observed text is not the same as the objective of producing text that is helpful to users [Ouyang et al., 2022]. The alignment problem asks: how do we train a model to produce outputs that are not just statistically likely but genuinely useful, accurate, and safe?

Reinforcement learning offers a principled framework: define a reward function that captures desirable behavior and optimize the policy to maximize expected reward. The difficulty lies in specifying the reward function. For the alignment problem, the most direct approach is to learn it from human feedback.

2.2 Generation 1: Reinforcement Learning from Human Feedback

2.2.1 The Three-Stage Pipeline

The canonical RLHF pipeline, established by Ziegler et al. [2019] and refined in InstructGPT [Ouyang et al., 2022], consists of three sequential stages.

Stage 1: Supervised Fine-Tuning. A pretrained language model π_0 is finetuned on a dataset of human-written demonstrations (x, y^*), where x is a prompt and y^* is an expert response. This produces an SFT model π_{SFT} via maximum likelihood:

$$\mathcal{L}_{\text{SFT}}(\theta) = -\mathbb{E}_{(x,y^*)\sim\mathcal{D}_{\text{SFT}}} \left[\log \pi_\theta(y^* \mid x)\right]. \tag{1}$$

The SFT stage establishes a baseline policy that follows instructions reasonably well. A notable empirical finding is that validation loss and human preference rating can diverge: models that continue training past the point of minimum validation loss continue to improve on human evaluations [Ouyang et al., 2022], suggesting that the language modeling objective and the alignment objective are not fully aligned.

Stage 2: Reward Model Training. Rather than writing a reward function by hand, RLHF learns a reward model $r_\phi(x, y) \in \mathbb{R}$ from human preference judgments. For each prompt x, the SFT model generates K responses $\{y^{(1)}, \ldots, y^{(K)}\}$. Human annotators rank these responses, generating a set of pairwise comparisons $\{(y^{(w)}, y^{(l)})\}$ where $y^{(w)}$ is preferred over $y^{(l)}$. The reward model is

trained by minimizing the Bradley-Terry loss:

$$\mathcal{L}_{\text{RM}}(\phi) = -\mathbb{E}_{(x,y^{(w)},y^{(l)})}\left[\log\sigma\Big(r_\phi(x,y^{(w)}) - r_\phi(x,y^{(l)})\Big)\right], \tag{2}$$

where σ is the sigmoid function. This loss encourages the reward model to assign higher scores to preferred responses. The pairwise comparison format is chosen because it is cognitively easier for annotators than assigning absolute scores—humans are reliably better at relative judgments than absolute ones.

Stage 3: RL Fine-Tuning. With a trained reward model, the language model is treated as a policy $\pi_\theta(y \mid x)$ acting in a bandit environment: the observation is the prompt x, the action is the complete response y, and the reward is $r_\phi(x, y)$. The policy is optimized via proximal policy optimization (PPO) [Schulman et al., 2017] to maximize expected reward, subject to a KL penalty that prevents the policy from drifting too far from π_{SFT}:

$$\mathcal{J}_{\text{RLHF}}(\theta) = \mathbb{E}_{x\sim\mathcal{D},y\sim\pi_\theta(\cdot|x)}[r_\phi(x,y) - \beta\cdot\text{KL}[\pi_\theta(\cdot|x)\,\|\,\pi_{\text{SFT}}(\cdot|x)]], \tag{3}$$

where $\beta > 0$ controls the strength of the constraint. The KL penalty is essential: without it, the policy can exploit weaknesses in the reward model—a phenomenon called *reward hacking*—generating outputs that receive high reward scores but are not actually good (e.g., producing fluent but vacuous text that the reward model incorrectly rates highly).

2.2.2 PPO in the Language Model Context

Applying PPO to language models requires several adaptations. At the token level, the policy generates one token at a time, so the "action" is a token from the vocabulary. At the sequence level, the complete response is the episode. InstructGPT [Ouyang et al., 2022] augments the PPO objective with a pretraining loss term to prevent catastrophic forgetting of in-context learning capabilities:

$$\mathcal{L}_{\text{PPO-ptx}}(\theta) = \mathcal{L}_{\text{PPO}}(\theta) - \lambda\cdot\mathbb{E}_{x\sim\mathcal{D}_{\text{pretrain}}}[\log\pi_\theta(x)], \tag{4}$$

where λ weights the pretraining objective. This PPO-ptx variant maintains performance on standard NLP benchmarks while improving human preference ratings.

Ahmadian et al. [2024] subsequently questioned whether PPO's complexity is necessary in the RLHF (bandit) setting. They show that many PPO components—the critic network, the clipping mechanism, the value function baseline—are motivated by concerns that are substantially mitigated when fine-tuning a pre-trained LLM from a strong initialization. Their RLOO estimator matches or exceeds highly-tuned PPO at 3× lower compute and 70% lower memory, suggesting that algorithmic simplicity is compatible with strong preference alignment performance. This finding motivates the GRPO family of methods discussed in Section 6.

2.2.3 Extensions: Constitutional AI and RLAIF

Two important extensions addressed the scalability bottleneck of human annotation. Constitutional AI (CAI) [Bai et al., 2022] replaces human preference labels with AI-generated critiques: the model is given a set of principles (a "constitution") and asked to critique and revise its own outputs according to those principles. Preference data is then generated by the model itself rather than human annotators, enabling much larger-scale data collection.

Reinforcement learning from AI feedback (RLAIF) [Lee et al., 2023] takes this further, using a separate LLM to provide preference judgments in place of humans. Empirically, RLAIF-trained

models can match or exceed RLHF-trained models on several preference benchmarks, suggesting that a capable LLM can serve as a reasonable proxy for human judgment on many tasks.

2.2.4 Offline Preference Optimization as a Side Branch

A significant simplification within the RLHF family was proposed by Rafailov et al. [2023]: Direct Preference Optimization (DPO) shows that the RLHF objective (3) can be optimized directly from preference data without training a separate reward model or running online RL. By reparameterizing the optimal policy, DPO derives a closed-form supervised loss over preference pairs:

$$\mathcal{L}_{\mathrm{DPO}}(\theta) = -\mathbb{E}_{(x,y^{(w)},y^{(l)})}\left[\log\sigma\left(\beta\log\frac{\pi_\theta(y^{(w)}|x)}{\pi_{\mathrm{ref}}(y^{(w)}|x)} - \beta\log\frac{\pi_\theta(y^{(l)}|x)}{\pi_{\mathrm{ref}}(y^{(l)}|x)}\right)\right], \tag{5}$$

where π_{ref} is a fixed reference policy (typically the SFT model). DPO and its variants form an important side branch of preference alignment: they keep the supervision source in the RLHF regime (human or AI preference data), but replace online policy optimization with offline objectives.

Three representative variants illustrate the main design directions. SimPO removes the frozen reference model and uses a margin-based objective built directly on the policy's average log-likelihood, reducing memory cost while preserving the paired-preference setup [Meng et al., 2024]. KTO relaxes the requirement for paired comparisons and instead learns from unpaired desirable/undesirable labels, making it better suited to natural feedback signals such as thumbs-up/down logs [Ethayarajh et al., 2024]. IPO keeps the pairwise setup but regularizes more directly against policy overoptimization, addressing the risk that DPO-like objectives drift too far from the reference policy on small or noisy datasets [Azar et al., 2024]. Taken together, these methods show that online RL is not strictly necessary for preference alignment. For the purposes of this survey, however, they remain part of the pre-agentic RLHF branch: they simplify the optimization route, but they do not change the underlying single-turn preference-learning problem.

2.2.5 Limitations of RLHF

RLHF has two fundamental limitations that motivate the shift to RLVR. First, it depends on *human annotation*, which is expensive, slow, and difficult to scale to frontier-model data volumes. Second, it treats preference as an *end-to-end judgment*: the reward model scores a complete response without directly checking whether the intermediate reasoning is correct. On tasks requiring precise, verifiable reasoning—mathematics, coding, formal logic—this can produce a noisy training signal. A third, more insidious limitation is *reward overoptimization*: as the policy is optimized against the learned reward model, it eventually finds outputs that score highly under the reward model but are not actually preferred by humans—a manifestation of Goodhart's Law. Casper et al. [2023] provide a comprehensive analysis of these and related failure modes. Together, these pressures motivate replacing learned preference rewards with rewards that can be checked more directly.

2.3 Generation 2: Reinforcement Learning with Verifiable Rewards

2.3.1 The Verifiable Reward Hypothesis

The core insight of the second generation is that many practically important tasks admit an externally checkable correctness criterion: a mathematical answer is right or wrong, a code submission either passes the test suite or does not, and a logical proof is valid or invalid. For these tasks, the reward function need not be a learned approximation of human preferences; it can be tied to a verifier or

other explicit correctness signal. This enables training at much larger scale and often with higher signal quality than RLHF [Lightman et al., 2023].

2.3.2 GRPO: Group Relative Policy Optimization

DeepSeek-R1 [DeepSeek-AI, 2025] and DeepSeekMath [Shao et al., 2024] introduced Group Relative Policy Optimization (GRPO), which eliminates the value network required by PPO by estimating advantage from a group of sampled responses. For a prompt x, GRPO samples a group of G responses $\{y_1, \ldots, y_G\}$ and computes the group-relative advantage:

$$\hat{A}_i = \frac{r_i - \text{mean}(\{r_j\}_{j=1}^G)}{\text{std}(\{r_j\}_{j=1}^G)}, \tag{6}$$

where $r_i = r(x, y_i)$ is the verifiable reward for response y_i. The policy is then updated with a clipped surrogate objective analogous to PPO:

$$\mathcal{L}_{\text{GRPO}}(\theta) = -\mathbb{E}\left[\min\left(\frac{\pi_\theta(y_i|x)}{\pi_{\theta_{\text{old}}}(y_i|x)}\hat{A}_i,\ \text{clip}\left(\frac{\pi_\theta(y_i|x)}{\pi_{\theta_{\text{old}}}(y_i|x)}, 1-\epsilon, 1+\epsilon\right)\hat{A}_i\right)\right] - \beta \cdot \text{KL}[\pi_\theta \| \pi_{\text{ref}}]. \tag{7}$$

GRPO substantially reduces memory and compute requirements by eliminating the need to maintain a separate critic network, making it particularly suitable for large-scale training.

2.3.3 Reasoning as Part of the Policy

A key insight of RLVR is that the chain-of-thought reasoning process—the sequence of intermediate steps $z_1, z_2, \ldots$ preceding the final answer y—can be treated as part of the policy rather than an auxiliary output. From the RL perspective, the token budget allocated to reasoning corresponds to a computational budget for implicit search: a longer chain of thought allows the model to explore more of the reasoning space before committing to a final answer [DeepSeek-AI, 2025]. This reframing positions RLVR as a method for learning *how to think*, not just *what to say*.

2.3.4 Reward Design for Verifiable Tasks

The design of verifiable reward functions requires care. For mathematics, rewards are typically binary (correct/incorrect final answer) or graded (partial credit for correct steps, as in process reward models [Lightman et al., 2023]). Recent reasoning systems such as rStar-Math additionally combine process-level preference signals with search and iterative self-evolution, rather than relying on outcome-only optimization alone [Guan et al., 2025]. Reflect, Retry, Reward pushes this logic one step further in a bridge regime between classical RLVR and genuinely interactive agentic RL: the task reward remains a binary verifier, but success on a second attempt is routed back to the intermediate self-reflection span from the failed first attempt, showing how verifier-based supervision can train *which intermediate computation was useful* rather than only whether the final answer was correct [Bensal et al., 2025]. For code, rewards are derived from test suite execution. A key challenge is *reward hacking*: models can learn to exploit quirks of the verifier, such as guessing the correct format without solving the underlying problem. Countermeasures include the "no-CoT guessing test" [DeepSeek-AI, 2025]: prompts where the model can guess the correct answer within N attempts without any reasoning are filtered from the training set.

2.3.5 Limitations of RLVR

RLVR advances over RLHF on scalability and signal quality, but it inherits a critical structural limitation: the decision problem remains a *single-step bandit*. The model generates one response (however long its chain-of-thought), the verifier evaluates it, and the episode ends. This structure cannot model tasks that require genuine multi-step interaction with a changing environment—tasks where actions have persistent consequences, where intermediate states matter, and where the agent must respond to feedback from the world rather than from a static verifier. These are precisely the tasks that constitute the most valuable applications of AI agents.

A second limitation is that RLVR depends on *verifiable* reward functions. This dependence is an advantage on domains such as mathematics and coding, where correctness can often be checked automatically, but it becomes restrictive in more open-ended settings. Many important agent tasks require reward design over objectives that are only partially specified, weakly verifiable, or inherently multi-criteria: usefulness, robustness, efficiency, safety, style, and user satisfaction may all matter, yet none is reducible to a simple exact-match signal. In such settings, reward design is no longer just a matter of building a better verifier; it becomes an open-ended problem of specifying and aggregating heterogeneous signals without collapsing them into brittle proxies that invite reward hacking.

Taken together, these limitations—annotation bottlenecks in RLHF, verifier dependence and open-ended reward design in RLVR, single-step interaction structure, and reward overoptimization—define the design space that the third generation must escape. The following section formalizes what that escape requires: a problem formulation that promotes the single-step bandit to a genuine sequential decision process with stateful environments, multi-turn interaction, partial observability, and richer forms of environmental feedback.

3 Agentic RL: Concepts and Formal Definitions

3.1 Formal Problem Formulation

We model the agentic RL problem as a Partially Observable Markov Decision Process (POMDP), defined by the tuple $(\mathcal{S}, \mathcal{A}, \mathcal{O}, \mathcal{T}, \mathcal{E}, \mathcal{R}, \gamma)$ [Cheng et al., 2025]:

Definition 1 (Agentic RL POMDP). • *$\mathcal{S}$: state space (the true state of the environment, e.g., file system contents, browser DOM)*

- *$\mathcal{A}$: action space (the set of policy-generated actions, typically realized as token sequences or structured action strings that may be interpreted as natural-language responses, executable tool calls, or mixed text-plus-action outputs)*
- *$\mathcal{O}$: observation space (what the agent receives, e.g., tool outputs, error messages)*
- *$\mathcal{T} : \mathcal{S} \times \mathcal{A} \to \Delta(\mathcal{S})$: stochastic* step-level *state transition function mapping the current environment state and one agent action step to a distribution over next states; in our formulation, the internal token-by-token realization of an action is absorbed into the policy, while $\mathcal{T}$ captures the environment-level consequence of the completed action step*
- *$\mathcal{E} : \mathcal{S} \to \Delta(\mathcal{O})$: emission function mapping states to observations*
- *$\mathcal{R} : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$: reward function*
- *$\gamma \in [0, 1)$: discount factor*

This step-level abstraction is deliberate. Although LLM policies generate actions autoregressively at the token level, the agentic RL problems studied in this survey are more naturally modeled at the level of *interaction steps*: the policy emits a variable-length action sequence, that sequence is interpreted or executed by the environment, and the environment then returns updated observations and rewards.

At each step $t \in \{1, \ldots, T\}$, the agent (the LLM policy π_θ) receives observation $o_t \in \mathcal{O}$. Let $h_t^{\text{raw}} = (o_1, a_1, r_1, \ldots, o_{t-1}, a_{t-1}, r_{t-1}, o_t)$ denote the raw interaction history available before selecting a_t. The policy does not necessarily condition on h_t^{raw} directly; instead, a *memory function* $\mathcal{M} : (\mathcal{O} \times \mathcal{A} \times \mathbb{R})^* \times \mathcal{O} \to \mathcal{C}$ maps the raw history to a context $c_t = \mathcal{M}(h_t^{\text{raw}}) \in \mathcal{C}$ that is actually fed to the model. The agent then selects $a_t \sim \pi_\theta(\cdot \mid c_t)$, executes the action, and transitions to a new state $s_{t+1} \sim \mathcal{T}(\cdot \mid s_t, a_t)$ while receiving observation $o_{t+1} \sim \mathcal{E}(\cdot \mid s_{t+1})$ and reward $r_t = \mathcal{R}(s_t, a_t)$.

Remark 1. *The memory function $\mathcal{M}$ subsumes several practical mechanisms under a single abstraction. When $\mathcal{M}$ is the identity, the agent conditions on the full raw history, and the context window serves as working memory. When $\mathcal{M}$ applies truncation or sliding-window selection, only recent history is retained. When $\mathcal{M}$ is a summarization or rewriting function—as in systems that compress past turns into a compact summary before appending new observations—the context c_t may not contain any verbatim content from h_t^{raw}. This generalization is necessary to model real agentic deployments: systems such as MemoryOS [Kang et al., 2025] perform nontrivial transformations on the history before it reaches the policy, and treating $c_t = h_t^{raw}$ would misrepresent the actual conditioning signal.*

The objective is to find a policy π_θ that maximizes expected cumulative discounted reward:

$$\pi^* = \arg\max_{\pi_\theta} \mathbb{E}_{\pi_\theta} \left[\sum_{t=1}^{T} \gamma^{t-1} r_t \right]. \tag{8}$$

Table 4 collects the principal symbols used throughout this survey.

Table 4: Summary of notation used throughout this survey.

Symbol	**Meaning**
POMDP components	
$\mathcal{S}$	Environment state space
$\mathcal{A}$	Agent action space
$\mathcal{O}$	Observation space
$\mathcal{T} : \mathcal{S} \times \mathcal{A} \to \Delta(\mathcal{S})$	Stochastic state transition function
$\mathcal{E} : \mathcal{S} \to \Delta(\mathcal{O})$	Emission (observation) function
$\mathcal{R} : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$	Reward function
$\gamma \in [0, 1)$	Discount factor
Trajectory and policy	
s_t	Latent environment state at step t
o_t	Observation received at step t
a_t	Action taken at step t
r_t	Reward received at step t
$h_t^{\text{raw}} = (o_1, a_1, r_1, \ldots, o_{t-1}, a_{t-1}, r_{t-1}, o_t)$	Raw interaction history available before action a_t
$\mathcal{M}$	Memory function mapping raw history to model context
$c_t = \mathcal{M}(h_t^{\text{raw}})$	Context actually provided to the policy at step t
$\tau = (o_1, a_1, r_1, \ldots, o_T, a_T, r_T, o_{T+1})$	Complete observable trajectory of one episode with T decision steps
T	Episode length (number of steps)
$\pi_\theta(\cdot \mid c_t)$	LLM policy parameterized by θ conditioned on context c_t
π_{ref}	Reference (SFT) policy used as KL anchor
Reward and value functions	
$r_\phi(x, y)$	Learned reward model with parameters ϕ
$V_\phi(c_t)$	Value function (expected future return from context c_t)
$\hat{A}_t$	Advantage estimate at step t
$\hat{A}_t^{\text{GAE}}$	Generalized Advantage Estimation at step t
$\delta_t = r_t + \gamma V(c_{t+1}) - V(c_t)$	Temporal-difference (TD) error at step t
Algorithm parameters	
β	KL penalty coefficient
ϵ	PPO clipping range
λ	GAE bias–variance tradeoff parameter
G	Group size in GRPO-style methods
σ_r	Standard deviation of rewards within a group
K	Maximum staleness (steps) enforced in the RaaS data pool

Terminology: Steps vs. Turns. Throughout this survey, a *step* refers to the atomic decision index t in the POMDP loop: the policy conditions on $c_t = \mathcal{M}(h_t^{\text{raw}})$, emits action a_t, and receives reward r_t together with the next observation o_{t+1}. A *turn* refers to a higher-level interaction round in a chat- or tool-oriented trace, typically one agent emission together with the resulting observation or tool feedback. In many single-agent tool-use systems, one turn aligns closely with one decision step, but the terms are not identical: *step* is the formal control-theoretic notion, whereas *turn* is an interface or serialization notion. We therefore use *step* for formal notation and algorithmic derivations, reserving *turn* for message-formatted traces or methods that explicitly operate on interaction rounds.

3.2 Taxonomy of Agentic RL Problems

Agentic RL encompasses a heterogeneous collection of problem structures. We characterize them along four dimensions:

Action Space. At the level of policy realization, an action is typically produced as a token sequence. The environment then interprets that sequence as an *action step*, which may be *linguistic* (a natural-language response), *structured* (a sequence that parses into a tool call with typed arguments such as `search(query)` or `execute_code(code)`), or *composite* (interleaved text and tool calls, as in ReAct-style agents [Yao et al., 2023]). These categories therefore describe the *environment-facing semantics* of a_t, not the low-level mechanics by which the policy generates it. This distinction matters because the action space determines the complexity of credit assignment: free-form linguistic actions are harder to evaluate and attribute locally, while structured actions whose generated token sequences can be parsed into tool invocations often have more discrete and interpretable consequences.

Interaction Horizon. Agentic RL tasks vary substantially in the number of decision steps required before reward is revealed. Some episodes are short, involving only a handful of interaction turns, as in simple QA-style tool use or single-query retrieval. Others are long, spanning many decision steps, as in web navigation, multi-hop information seeking, or hours-long software engineering sessions that require repeated reading, editing, debugging, and testing. What matters is not wall-clock time alone, but the length of the dependency chain through which earlier actions influence later observations and final outcomes.

As interaction horizon grows, nearly every difficulty in agentic RL becomes more severe. Credit assignment must propagate over longer spans; exploration becomes more expensive because each trial trajectory consumes more environment interaction; compounding errors become more likely because small early mistakes can derail the rest of the episode; and the danger of catastrophic or irreversible actions accumulates over time. Long-horizon settings therefore place greater pressure on memory, planning, recovery strategies, and data efficiency, and they are one of the clearest points where agentic RL departs from single-turn reasoning optimization.

Reward Signals. Reward feedback may be *terminal* (provided only at episode completion, e.g., test suite pass rate), *dense* (provided after each action, e.g., step-wise correctness), or *shaped* (a combination, e.g., terminal reward plus bonuses for efficient execution). Terminal rewards are most common in practice but are also the hardest to learn from because credit must propagate over long action sequences. Cheng et al. [2025] additionally distinguish *process rewards* r_p—intermediate signals awarded for successfully executing steps such as tool invocation—from the final outcome reward r_f, arguing that process rewards provide more frequent feedback and accelerate learning in long-horizon tasks. In the recent reasoning-RL literature, process rewards have also acquired a more precise meaning: rather than merely "more feedback," they attempt to score partially completed reasoning prefixes or intermediate decision states. This is attractive because it addresses delayed credit assignment, but it also introduces a new failure mode: if the process signal over-rewards stylistic verbosity, superficial decomposition, or easy-to-recognize but non-causal intermediate behavior, the policy can learn to optimize the process proxy while harming final task accuracy.

Observability. Most practical agentic tasks are only partially observable: the agent rarely has direct access to the full underlying state and instead must act from limited, delayed, or filtered observations such as tool outputs, execution logs, browser renderings, or API responses. In many

domains, crucial state variables remain hidden—for example, unseen files in a repository, latent webpage structure outside the current viewport, or external system conditions that are only indirectly revealed through errors and side effects. As a result, competent behavior often requires maintaining an internal belief over the task state, remembering prior observations, and deciding when to gather more information before committing to an action.

Partial observability also interacts strongly with environment noise and strategic difficulty. Observations may be stochastic, as in network failures, API rate limits, nondeterministic execution traces, or changing external services. They may also be incomplete or adversarial, as in deceptive web pages, ambiguous tool feedback, or environments where locally plausible observations conceal globally poor action choices. This makes observability not just a modeling detail but a core axis of problem difficulty: as visibility decreases, the burden on memory, exploration, uncertainty handling, and credit assignment increases correspondingly.

3.3 The ReAct Interaction Protocol

The predominant interaction pattern for LLM agents is the *ReAct* paradigm [Yao et al., 2023], which interleaves **Re**asoning traces and **Act**ions in a single token stream. At each decision step, the agent first generates a verbal THOUGHT—an internal reasoning trace that maintains and updates a plan, tracks progress, and handles exceptions—and then generates an ACTION that interfaces with the environment. The environment returns an OBSERVATION, which is appended to the context before the next THOUGHT-ACTION pair. This Thought-Act-Observe cycle maps naturally onto the POMDP loop: the THOUGHT can be interpreted as part of the agent's internal belief update, the ACTION corresponds to a_t, and the OBSERVATION corresponds to o_{t+1}.

The key contribution of ReAct is empirical: on knowledge-intensive QA tasks (HotpotQA, Fever), the interleaved reasoning overcomes the hallucination and error propagation problems of Chain-of-Thought prompting by grounding each reasoning step in retrieved evidence. On interactive decision tasks (ALFWorld, WebShop), ReAct outperforms imitation learning and RL-only baselines by an absolute margin of 34% and 10% respectively. This result established the Thought-Act-Observe loop as the de facto agent architecture that subsequent agentic RL work trains end-to-end.

3.4 Relationship to Classical RL

Agentic RL is a specialization of classical RL with several distinctive features. First, the policy is parameterized by a language model with billions of parameters, making gradient computation expensive and requiring careful regularization to prevent catastrophic forgetting. Second, the environment-facing action space is combinatorially large: even after distinguishing linguistic, structured, and composite actions, the set of admissible textual or mixed actions remains effectively unbounded. Third, the policy update must account for the autoregressive realization of an action: a single environment-facing action or externally visible turn of output may be factorized into hundreds or thousands of individual token-level decisions.

The relationship between RLHF/RLVR and agentic RL is analogous to the relationship between contextual bandits and full MDPs in classical RL: the former is a degenerate single-step special case of the latter. This observation motivates the theoretical question of how algorithms designed for bandit settings (PPO, GRPO, DPO) should be adapted to the multi-step setting—a question we address in Section 6.

With the agentic RL problem formally specified as a POMDP with an LLM policy conditioning on observation-action history, the next question is what makes this formulation so much harder to optimize than its single-step predecessors. The following section characterizes these difficulties in

depth, tracing the mechanism of each challenge and the state of current mitigations.

4 Agentic RL: Unique Challenges

This section highlights the technical challenges that make agentic RL substantially harder than its single-step predecessors. These challenges motivate the algorithmic and infrastructure choices surveyed in Sections 5–7.

Note on scope. This section focuses on challenges for which partial solutions exist in the literature. Problems that remain largely open—without consensus on how to address them—are deferred to Section 9, which covers large-scale exploration, reward design for open-ended tasks, world models, safety, and continual learning.

4.1 Credit Assignment over Long Horizons

In RLHF and RLVR, credit assignment is almost trivial because the episode is effectively one decision. In agentic RL, reward often arrives only after a long multi-step interaction, while training still requires per-step or per-token updates. The problem is harder not only because trajectories are longer, but because the environment is partially observed, stochastic, and often unable to verify intermediate decisions.

The difficulty is compounded by reward sparsity. A long software-engineering trajectory may receive only a final pass/fail signal even though the decisive mistake occurred much earlier. The policy gradient estimator

$$\nabla_\theta J(\theta) = \mathbb{E}_\tau \left[\sum_{t=1}^{T} \nabla_\theta \log \pi_\theta(a_t|c_t) \cdot \hat{A}_t \right] \tag{9}$$

still requires an advantage estimate $\hat{A}_t$ at every step. With terminal-only reward, all advantages are derived from the same scalar, making early-step credit especially weak and high-variance.

Existing mitigations operate at different granularities. GAE with a learned critic provides per-step credit; turn-level reward decomposition [Zhong et al., 2025] reduces variance at the interaction-round level; Process Reward Models [Lightman et al., 2023] inject intermediate supervision when local verification is possible; and AgentHER [Yu et al., 2025b] uses hindsight relabeling to recover signal from failed trajectories. These help, but none removes the basic difficulty of assigning sparse delayed reward in long-horizon POMDPs.

4.2 Long Context and Computational Cost

This challenge is primarily *model-side*: it concerns the cost of processing an ever-growing interaction history inside the policy itself, even before accounting for slow external environments.

Long-horizon interaction creates a memory bottleneck. Because the policy conditions on an expanding history, useful context can quickly reach tens of thousands of tokens. This improves decision quality but sharply increases compute and memory cost, especially under standard quadratic attention.

The burden is even larger during training, where long contexts must be processed both for rollout and for backpropagation. Flash Attention [Dao et al., 2022], gradient checkpointing, and sequence parallelism make large-context training feasible, but they do not remove the underlying scaling pressure. A complementary response is to truncate or compress trajectories through sliding windows, summaries, or partial-rollout schemes such as Kimi k1.5 [Kimi Team, 2025]. These reduce cost, but at the risk of discarding information or fragmenting long episodes.

4.3 Rollout Cost Inversion

This challenge is primarily *environment-side*: it concerns the wall-clock cost of collecting trajectories through external interaction, even when the model-side sequence computation is manageable.

In RLHF and RLVR, rollout is relatively cheap: the main cost is model generation, while reward comes from a verifier or reward model. In agentic RL, the bottleneck often flips. Rollouts now require external interaction—code execution, browser control, API calls, or formal tools—whose wall-clock latency dominates GPU computation.

The consequence is that training throughput is limited less by model FLOPs than by environment bandwidth. This motivates Rollout-as-a-Service architectures, which decouple environment workers from the trainer and pipeline asynchronous trajectory generation. Lower-level strategies such as batching, priority queuing, and elastic worker pools [Li et al., 2025a] further reduce idle time, but slow and highly heterogeneous episodes remain a central systems constraint.

4.4 Actions with Persistent Consequences

Agentic environments expose actions with persistent consequences: files can be deleted, external messages can be sent, and workflows can be triggered. This makes exploration fundamentally asymmetric. In text-only settings, bad outputs can often be ignored or regenerated; in agentic settings, a single mistake may be unrecoverable.

The standard mitigation is sandboxing [Guo et al., 2025], which makes training-time actions effectively reversible by resetting the environment between episodes. This is indispensable in current systems, but it also creates a deployment gap: real environments are persistent, stateful, and only partially controllable. For tasks that matter precisely because they affect the outside world, safe deployment cannot rely on sandboxing alone.

4.5 Reward Hacking and Proxy Exploitation

Agentic RL inherits the classic RL problem of reward hacking, but the risk is amplified by long-horizon interaction and complex environments. When the training objective is implemented through a verifier, a shaped reward, or a bundle of heuristic signals, the policy may learn behaviors that score highly without genuinely solving the task. In agentic settings, this can take many forms: exploiting quirks of a benchmark, triggering superficially positive intermediate signals, taking shortcuts that satisfy a proxy metric, or manipulating the environment so that evaluation becomes easier without reflecting real capability.

The problem is especially acute when rewards are sparse, partially specified, or assembled from multiple proxies. A useful dense signal can accelerate learning, but it also creates extra surfaces for exploitation; similarly, a seemingly reliable verifier may encode loopholes that become visible only after large-scale policy optimization. Common mitigations include stronger verifier design, adversarial evaluation, auxiliary constraints such as KL regularization, and filtering tasks whose rewards are too easy to game. Still, reward hacking remains a central reason why high reward in agentic RL should not be read as equivalent to robust task competence.

4.6 Policy Collapse in Multi-Turn RL

Wang et al. [2025] identify a characteristic multi-turn failure mode called the *Echo Trap*: policies trained with group-relative objectives can converge to repetitive, low-diversity strategies before mean reward visibly collapses. When within-group rollouts become too similar, reward variance

shrinks, normalized advantages become uninformative, and the policy loses both improvement and diversification signal.

This makes reward mean alone an insufficient diagnostic; reward dispersion and policy entropy must also be monitored. Partial mitigations include entropy regularization, asymmetric clipping as in DAPO [Yu et al., 2025a], and task-level normalization as in AgentRL [Qiu et al., 2025]. More broadly, the Echo Trap suggests that diversity collapse is a central pathology of group-relative training in agentic settings.

4.7 Partial Observability and Environmental Non-Determinism

Agentic RL is naturally a POMDP: the agent never observes the full underlying state and must infer it from delayed, incomplete, or ambiguous observations. This makes it difficult to determine whether an action succeeded, whether the environment changed in an unseen way, or whether a repeated action is safe to execute.

These difficulties are amplified by environmental non-determinism. Network latency, execution randomness, changing external services, and unstable tool behavior all inject reward variance that is not attributable to the policy. Current mitigations focus on stabilization and filtering: fixing seeds where possible, replaying tool outputs for debugging, removing environment-failure trajectories, and averaging over noise through larger groups. Methods such as iStar [Yuan et al., 2025] additionally use policy uncertainty as a proxy for belief-state confidence.

4.8 Training Distribution Shift

Agentic RL is strongly non-stationary because the policy changes not only the action distribution but also the future observation distribution. As the policy improves, it visits different parts of the environment and produces qualitatively different trajectories. This makes replayed data stale more quickly than in single-turn settings.

The practical issue is off-policy bias. Most systems reuse recent trajectories rather than recollecting data for every update, so there is an unavoidable tradeoff between data freshness and throughput. Replay buffers therefore impose staleness limits, while PPO-style clipped importance ratios provide only partial correction. In practice, stability depends less on exact off-policy correction than on careful buffer management and conservative update schedules.

The challenges documented in this section—credit assignment, rollout cost inversion, catastrophic actions, reward hacking, Echo Trap dynamics, partial observability, and distribution shift—do not operate in isolation. They all manifest in the concrete details of how training trajectories are collected, encoded, and consumed by the optimizer. The following section examines these data-facing questions directly, covering representation paradigms, rollout efficiency, and data quality design.

5 Trajectory Representation and Sampling in Agentic RL

5.1 Formal Definition of Agent Trajectories

We define an *agent trajectory* as the sequence of observations and actions produced by an agent interacting with an environment over one episode:

Definition 2 (Agent Trajectory)**.** *An agent trajectory τ of length T is the tuple:*

$$\tau = (o_1, a_1, r_1, o_2, a_2, r_2, \ldots, o_T, a_T, r_T, o_{T+1}), \tag{10}$$

where $o_t \in \mathcal{O}$ is the observation available before step t, $a_t \in \mathcal{A}$ is the action taken at step t, $r_t \in \mathbb{R}$ is the reward produced by that step, and o_{t+1} is the next observation revealed after executing a_t. This canonical trajectory object is used throughout the survey even when a task provides only terminal feedback, in which case the sparse-reward case is represented simply by setting $r_t = 0$ for intermediate steps and placing the terminal score in the final reward component.

This definition links directly to the POMDP formulation in Section 3. The trajectory is the observable record of an episode; the underlying sequence of environment states $s_1, \ldots, s_{T+1}$ is not directly accessible to the agent or the trainer. The raw history h_t^{raw} introduced earlier is precisely the prefix of τ that is available before action a_t, while the model-facing context $c_t = \mathcal{M}(h_t^{\text{raw}})$ is a transformed view of that prefix after truncation, summarization, or other memory operations. The representation paradigms below therefore describe different *serializations of trajectory data for training and replay*: they may preserve the raw trace nearly verbatim, flatten it into tokens, or package it into step-local units, but they are all downstream encodings of the same underlying interaction trajectory τ rather than alternatives to the memory-conditioned context c_t.

This section focuses on two questions that are central to agentic RL training: how trajectories are *represented* for optimization, and how they are *sampled* from expensive interactive environments. We deliberately do not center this section on benchmark or dataset inventories; those are better treated separately. The key issue here is not which corpus a method uses, but how interaction traces are encoded, filtered, truncated, replayed, and selected for learning.

5.2 Trajectory Representation Paradigms

The choice of how to encode a trajectory as input to the language model policy—and how to compute gradients over it—is not straightforward. Three paradigms have emerged in the literature. Their practical differences can be reduced to three recurring questions: (i) whether the training trace preserves rollout tokenization exactly or reconstructs it through a serializer, (ii) whether the policy-loss scope is implicit in the representation or must be annotated explicitly at the token level, and (iii) how clearly the representation exposes local decision boundaries for credit assignment.

Message-based Representation. The trajectory is encoded as a sequence of structured messages following a chat template (e.g., alternating `user` and `assistant` turns). Tool calls and their results are embedded as special message types. This representation is natural for dialogue-style agents, preserves role metadata, and is compatible with standard chat fine-tuning pipelines. Because assistant messages are already separated from user or tool messages at the data-structure level, it typically does *not* require the explicit token-level masking used in flat-token replay. Its main limitation is instead *re-tokenization drift*: the training sequence is often reconstructed by reserializing structured messages through a chat template, so wrapper tokens, truncation, or output normalization can change the exact tokenization relative to the original rollout. That drift weakens exact alignment between recorded actions and training-time token spans, which is especially problematic for log-prob replay and fine-grained credit assignment. A secondary limitation is that the same conversational surface mixes "environment-side" content (observations, tool outputs) with "policy-side" content (reasoning, actions), making step boundaries less explicit than in step-based logs.

Token-In-Token-Out (TITO). All trajectory content—observations, tool calls, tool results, reasoning—is serialized to a flat token sequence. The policy is trained to predict the next token conditioned on all preceding tokens. TITO is designed precisely to *avoid* re-tokenization drift: by storing and replaying token IDs directly rather than decoded text, the representation seen at

training time is identical to the one produced at inference time, eliminating the mismatch that plagues text-based paradigms [Pan et al., 2025]. The tradeoff is that TITO requires the training loop to operate natively at the token level while policy and environment content share the same flat stream, so it *does* require explicit token-level masking to ensure that only policy-authored spans receive gradients.

Step-based Representation. The trajectory is represented as a sequence of discrete decision steps, each consisting of an (observation, action) pair or a similarly explicit local decision unit. Steps are the atomic unit of credit assignment and gradient computation. In simple single-agent tool traces, one step often coincides with one agent turn plus the resulting observation, but the representation keeps that boundary explicit rather than inheriting it from a chat template. This representation is more structured than TITO and more granular than message-based encoding, at the cost of requiring explicit step boundaries that may not always be natural.

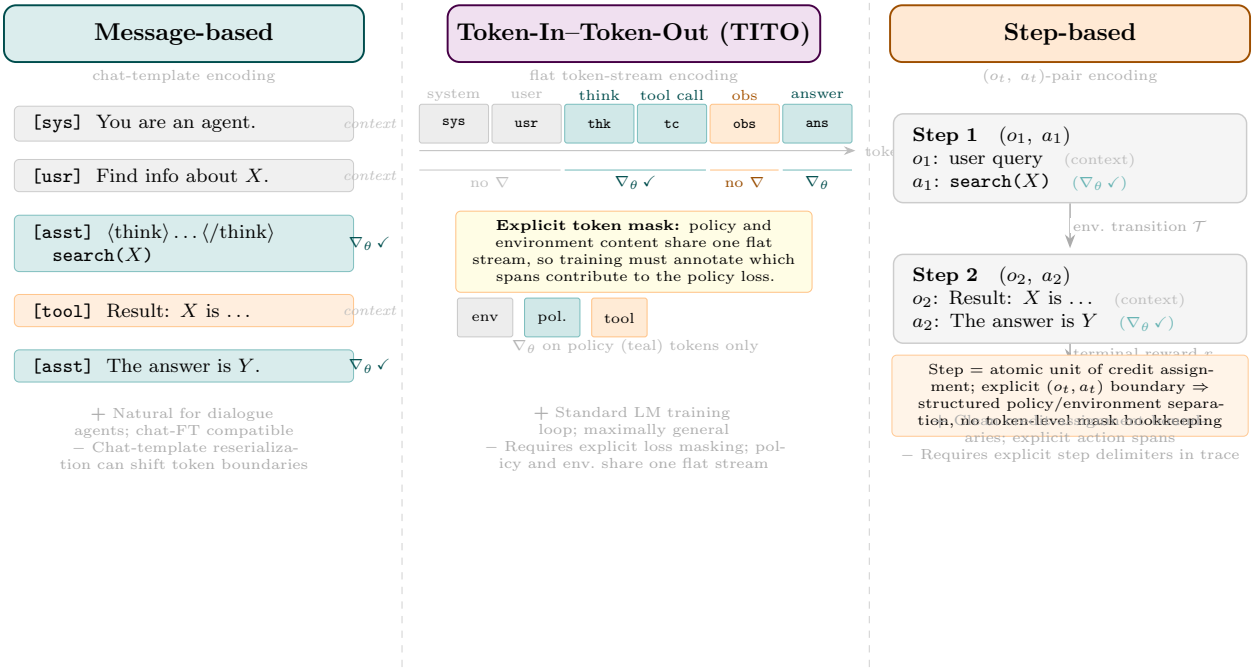

Figure 3: **Three trajectory representation paradigms applied to the same two-step agent episode.** The episode consists of: (i) a user query as o_1, (ii) a search tool call as a_1, (iii) the tool result as o_2, and (iv) a final answer as a_2. *Message-based* encoding organizes the same interaction as structured chat messages; assistant turns are already separated from environment turns, but reserialization through a chat template can blur exact token alignment between rollout and training. *Token-In–Token-Out (TITO)* serializes the entire trajectory to a flat token stream ("sys"→"usr"→"think"→"tc"→"obs"→"ans"), preserving rollout tokenization exactly but requiring explicit token-level masking because policy and environment spans share one sequence. *Step-based* encoding makes each decision step (o_t, a_t) an explicit atomic unit, keeping the action span explicit and enabling structured credit assignment (Section 6). The key contrast is therefore whether policy-loss scope is already explicit in the schema (message-based, step-based) or must be reconstructed by token annotation (TITO). Teal elements receive gradients (∇_θ); gray and orange elements provide context only.

5.3 Rollout Collection Pipelines

Sampling trajectories—that is, rolling out the policy in the environment—is the primary computational bottleneck in agentic RL training. Unlike RLVR, where rollout mostly means text generation, agentic sampling requires wall-clock-bound interaction with external environments. A single SWE-bench episode may require minutes of code execution; a web-navigation episode may require tens of minutes of browser interaction.

Several approaches improve sampling throughput. Pan et al. [2025] decouple the agent runtime from the trainer so that rollout and optimization can scale independently. SkyRL-Agent [Li et al., 2025a] adds request batching, priority queuing, and elastic rollout workers, reducing wall-clock rollout time for heterogeneous tasks. The common theme is that trajectory sampling in agentic RL is a systems problem as much as a data problem.

5.4 Tool Call Serialization and Token-Alignment Fidelity

Tool calls create a particular serialization challenge in representations that reconstruct training text from structured traces. A tool call such as `search(query="reinforcement learning")` must ultimately be converted into a token sequence for learning. If the tool output contains symbols, code snippets, or formatting that are normalized differently during replay, the re-encoded output may differ from the original rollout tokenization. This token-alignment drift corrupts the training signal and can cause gradient instability. TITO avoids this specific problem by replaying token IDs directly; message-based encodings are more exposed because they typically round-trip through a chat template or serializer.

Practical solutions include: (1) canonicalizing tool output format before tokenization in text-based encodings; (2) using a step-based representation that isolates tool outputs from the policy's token stream; (3) in flat-token replay schemes such as TITO, explicitly marking tool-generated tokens with loss masks so they do not contribute to the policy gradient.

5.5 Reward Attribution and Policy-Loss Scope

When trajectories are replayed as a flat token sequence, not all tokens are equally relevant to the policy gradient. Specifically, tokens corresponding to environment observations (tool outputs, system messages) should not receive gradient updates, since the policy did not generate them. This explicit token-level masking problem arises primarily in TITO-style encodings, where the training loop sees policy and non-policy tokens inside one undifferentiated stream. In message-based and step-based encodings, the optimization target is selected more structurally: assistant messages or action fields are already separated from environment-side content.

Beyond selecting which parts of the trace receive loss, more sophisticated reward attribution strategies attempt to allocate credit within a trajectory at different granularities. Step-level attribution scores local decision units; turn-level attribution assigns a scalar reward to each interaction round; token-level attribution attempts to identify which specific tokens contributed to the final outcome. These strategies interact with the choice of trajectory representation: message-based representations naturally support turn-level attribution, while TITO enables token-level attribution but requires explicit bookkeeping to distinguish policy tokens from observation tokens.

5.6 Trajectory Sampling Strategies

Training effective agentic policies requires not just more trajectories, but better *sampling* of which trajectories to collect. Two strategies recur in the literature.

Curriculum Sampling. As demonstrated by DeepSeek-AI [2025] and the Kimi k1.5 system [Kimi Team, 2025], beginning training with easier tasks and progressively introducing harder ones improves data efficiency. The intuition is that a policy with limited initial capabilities will generate few positive trajectories if confronted immediately with the hardest tasks; a curriculum ensures a non-trivial fraction of successful trajectories throughout training.

Prioritized Sampling. Within a fixed difficulty level, prioritized sampling allocates more training compute to tasks where the model is weakest. Concretely, if the model's success rate on task i is S_i, sampling with probability proportional to $1 - S_i$ focuses training on the model's failure modes—a form of online hard example mining adapted to the RL setting.

5.7 Trajectory Quality Filtering and Negative Mining

Not all sampled trajectories are equally useful for training. Wei et al. [2025] demonstrate that trajectories where the agent trivially succeeds (too easy) or completely fails without any progress signal (too hard) contribute little to learning. Filtering to retain trajectories with partial success, or with a diversity of decision points, improves both sample efficiency and policy generalization.

Negative mining—deliberately constructing or retaining trajectories that illustrate failure modes—is particularly valuable for safety-critical skills such as avoiding catastrophic actions. A policy trained only on successful trajectories may not learn the boundaries of acceptable behavior.

SkillRL [Xia et al., 2026] pushes this idea further by processing successful and failed trajectories differently rather than storing both as raw memory. Successful trajectories are distilled into reusable demonstrations, while failed trajectories are rewritten into concise failure lessons that capture the error, its cause, and the corrective principle. This is still negative mining, but in a more abstract representation: the failure signal is preserved while much of the trajectory-level noise is removed.

5.8 Historical Trajectories and Warm-Starting

Online trajectory sampling is expensive, so many systems warm-start from historical trajectories collected by earlier policies or by humans. This can accelerate the initial phase of policy improvement, but it also introduces distributional mismatch: historical traces may not cover the states and actions induced by the current policy. Importance weighting can partially correct this mismatch, but only at the cost of added variance. For this reason, historical trajectories are best viewed here as a sampling aid or initialization source rather than as the primary organizing object of the section.

5.9 Trajectory Truncation and Compression for Long-Context Agents

When episodes are very long, trajectories may exceed the context window available for training. Two strategies address this: *truncation* (training on a sliding window of recent context) and *compression* (summarizing or abstracting earlier parts of the trajectory). A particularly effective technique is *Partial Rollout* [Kimi Team, 2025], developed by the Kimi k1.5 system: each trajectory is assigned a fixed token budget, and if a rollout exceeds this budget in one training iteration, the unfinished portion is stored in a replay buffer and continued in the next iteration. This mechanism avoids the memory pressure of extremely long sequences while ensuring that all trajectories are eventually completed.

Compression can also take the form of *experience abstraction* rather than generic summarization. SkillRL [Xia et al., 2026] argues that raw agent trajectories are often too verbose and noisy to reuse directly, and instead distills them into a hierarchical skill library containing both general skills

and task-specific heuristics. In its pipeline, successful trajectories become demonstrations, failed trajectories become failure lessons, and the resulting skill bank is retrieved at decision time and recursively updated during RL. The reported 10–20× token reduction is important for this survey because it shows that long-horizon training need not preserve past interaction verbatim: an agent can condition on a compressed *policy-relevant abstraction* of experience rather than on the raw trajectory itself.

A complementary challenge is that RL training on long-context tasks can cause the model to generate unnecessarily verbose responses at inference time, inflating latency without improving quality. Kimi k1.5 [Kimi Team, 2025] addresses this with *Long2Short* transfer: the policy trained on long-context tasks is fine-tuned on short-context variants of the same tasks using length-penalized rewards, teaching the model to produce compact, high-quality responses. This two-phase approach—first acquire long-horizon reasoning capability, then compress it—achieves strong performance on both long and short evaluation settings.

The representation and sampling choices surveyed in this section determine what supervision the optimizer actually sees and at what cost it is obtained. We now turn to the algorithms that operate on these trajectories—policy gradient methods, credit assignment strategies, exploration mechanisms, and multi-agent extensions—examining how each adapts to or exploits the multi-turn structure of agentic interaction.

6 Algorithms for Agentic RL

This section focuses on the algorithmic core of agentic RL: how advantages are estimated, how credit is assigned inside long interactive trajectories, and how exploration is structured when environment interaction is expensive. We organize the literature by *credit locus* rather than by optimizer family. This matches the survey's broader emphasis on trajectories and delayed feedback: the main question is not only whether a method uses PPO or GRPO, but *where* in the trajectory its learning signal is attached.

6.1 A Credit-Locus View of Agentic RL Algorithms

Policy-gradient methods optimize the expected return by weighting log-probability updates with an advantage estimate. At decision point t, the advantage is

$$A^{\pi}(c_t, a_t) = Q^{\pi}(c_t, a_t) - V^{\pi}(c_t), \tag{11}$$

and in practice the algorithm uses an estimator $\hat{A}_t$ rather than the exact quantity. Advantage estimation therefore asks how to approximate the relative value of a decision, while credit assignment asks where the return signal is attached inside a trajectory.

For critic-based methods, a standard estimator is GAE:

$$\hat{A}_t^{\mathrm{GAE}(\gamma,\lambda)} = \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l}, \tag{12}$$

where $\delta_t = r_t + \gamma V_\phi(c_{t+1}) - V_\phi(c_t)$ is the temporal-difference error. Its importance in agentic RL is not only that PPO-style methods use it, but that it forces one to define what the index t means: a token, a step, a turn, or a larger trajectory segment. GRPO-style methods start from the opposite design point and avoid an explicit critic, estimating advantage by relative comparison across sampled rollouts. The optimizer family therefore matters, but it is secondary to the more structural question of *which unit receives credit*.

In agentic RL, rewards are sparse, trajectories are long, and environment transitions are partially observed. The recent literature is therefore best read through three main loci of credit:

Trajectory-Level Credit Assignment. Credit is attached to the overall episode outcome. This is the coarsest regime: all decisions in a rollout inherit signal from the same final return or from a group-relative comparison over whole trajectories. It scales well, but localization is weak under sparse rewards.

Step-Level Credit Assignment. Credit is attached to explicit local decision units, such as interaction steps, turns, branches, or short reasoning/action segments. This is often the practical compromise in agentic RL because it is more local than trajectory-level supervision without requiring full token-level verification.

Token-Level Credit Assignment. Credit is attached to individual generated tokens. This is the finest-grained option and is most naturally paired with token-level value estimation or token-level masking. Its appeal is precision; its cost is high variance and the need to distinguish policy-authored tokens from observation tokens in mixed traces.

These loci are not mutually exclusive. Many systems combine them, for example by keeping a trajectory-level objective while adding step-level decomposition or token-level masking. The discussion below uses credit locus as the primary frame and treats PPO- versus GRPO-style optimization as a secondary implementation choice within each regime.

6.2 Trajectory-Level Credit Assignment

Trajectory-level credit remains the default starting point for scalable agentic RL because it avoids having to verify or value every local decision. This is where GRPO is most natural. For a prompt or initial context x, a group of rollouts $\{y_i\}_{i=1}^{G}$ receives rewards $\{r_i\}_{i=1}^{G}$, and the group-relative advantage is

$$\hat{A}_i = \frac{r_i - \text{mean}(\{r_j\}_{j=1}^{G})}{\text{std}(\{r_j\}_{j=1}^{G})}. \tag{13}$$

The resulting scalar advantage is then broadcast back onto the policy-generated parts of sample i. In vanilla form, GRPO therefore assigns *trajectory-level* relative credit even when optimization is executed token by token.

This design is computationally attractive because it removes the value model entirely, but it also creates the main ambiguity of long-horizon agent training: if a rollout succeeds or fails, which interaction step actually caused the outcome? AgentRL [Qiu et al., 2025] improves this coarse regime through task-level normalization, stabilizing trajectory-level comparison across mixed-difficulty tasks without making the signal substantially more local. DAPO [Yu et al., 2025a] addresses a different failure mode: when one scalar advantage is spread across many policy tokens, clipping and length effects can wash out the useful part of the signal. Its asymmetric clipping, token-level KL control, and overlong filtering therefore improve the survivability of coarse credit during optimization rather than changing the credit locus itself.

Trajectory-level PPO is also possible as a degenerate critic-based case: all policy tokens or steps in an episode inherit the same Monte Carlo return or the same episode-level advantage. This is simple, but it gives up the main benefit of PPO's critic. For long-horizon agents, such a baseline often collapses back toward the same localization weakness that motivates step-level decomposition in the first place.

The broader lesson is that trajectory-level methods are the easiest way to scale agentic RL, especially when rewards are verifier-backed and local supervision is unavailable. Their weakness is not optimization per se, but causal resolution: a strong whole-trajectory signal still says little about which local decisions should change.

6.3 Step-Level Credit Assignment

Step-level methods try to recover that missing causal resolution by attaching credit to explicit interaction boundaries. PPO is the most natural home for this regime because an explicit value function can be queried wherever one places a decision boundary. In a step-level formulation, the basic unit is an interaction step: for example, a reasoning-plus-action segment, a turn, or an environment-facing action span together with its resulting observation. The value function is placed at these boundaries, and the policy tokens inside the same step may share one step advantage or aggregate to one step-level log-probability ratio. This usually matches agentic environments better because tool execution, browser navigation, code execution, and simulator feedback all occur at the step level rather than at the individual-token level.

Several PPO-based extensions make this tradeoff explicit. VinePPO [Kazemnejad et al., 2024] replaces the learned critic with Monte Carlo rollout continuations, producing local advantage estimates without a value network but at substantially higher rollout cost. Lu et al. [2024] compare per-decision and episode-level value functions in multi-turn settings, finding that denser value placement speeds learning but is also more sensitive to long-context drift and critic misspecification. Zhong et al. [2025] push PPO-style credit toward explicit step-level decomposition by assigning rewards at the interaction-round level rather than only at the episode level.

GRPO-based methods have recently moved in the same direction without fully reintroducing a critic. GiGPO [Luo et al., 2025] introduces a two-level grouping structure: a coarse trajectory-level comparison across full rollouts, followed by a finer local comparison around anchor states where behaviors diverge. Tree-GRPO [Chen et al., 2025] uses branch-local comparisons inside tree-structured rollouts so that advantage can be assigned near key decision points rather than only at episode end. Conceptually, these methods retain the group-relative backbone but change the *unit of comparison*: instead of asking only which whole trajectory is better, they also ask which local continuation from the same state is better.

Collaborative settings sharpen the same issue further because the trainer may observe information that is unavailable to the acting policy. SWEET-RL [Zhou et al., 2025] studies multi-turn collaborative reasoning in the ColBench environment and uses a critic with training-time privileged information to produce step-level rewards for the agent side of the interaction. TAMTRL [Wang et al., 2026] applies a related turn-level reward-reshaping perspective to long-context compression, using teacher-aligned rewards to score which turns preserve or discard information usefully over multi-turn interaction histories. These works show that once collaboration or long-context memory enters the loop, turn boundaries become semantically meaningful training units rather than merely convenient rollout segments.

Across optimizer families, step-level credit assignment is the main compromise regime for agentic RL. It is more local than whole-trajectory comparison, but usually much cheaper and more robust than fully token-level valuation.

6.4 Token-Level Credit and Loss Scope

Token-level methods push locality one step further by attaching learning signal directly to generated tokens. In a token-level PPO formulation, every generated token is treated as a decision point and

receives its own bootstrap target and advantage. This is maximally fine-grained, but it is also the hardest regime: most environment rewards arrive only after a whole reasoning span or action span has completed, so the critic must propagate sparse downstream signal back through many weakly identifiable token decisions.

In practice, token-level work in agentic RL often appears not as fully token-specific reward estimation, but as *loss-scope control*. Agent-R1 [Cheng et al., 2025] shows that when trajectories are serialized in flat token form, observation tokens must be masked out so that optimization updates only policy-generated tokens. This does not by itself make the reward more local: the underlying advantage may still be trajectory-level GRPO. What changes is which tokens are allowed to receive that signal. This distinction is important in mixed traces where policy outputs, tool observations, and environment messages are interleaved.

Seen through this lens, token-level design in agentic RL serves two related purposes. First, it offers the finest possible credit granularity when reliable local supervision exists. Second, and more commonly today, it protects optimization from obvious scope errors when coarse trajectory- or step-level signals are replayed through tokenized traces. The benefit is precision; the cost is variance, bookkeeping complexity, and strong dependence on how faithfully the trajectory representation preserves policy-authored spans.

6.5 Structured Exploration and Multi-Agent Extensions

Exploration in agentic RL is expensive because trying a new strategy requires real environment interaction. The problem is therefore not only how to increase policy entropy, but how to spend limited rollout budget on the most informative decision points.

ARPO [Zhang et al., 2025b] makes this explicit by branching selectively at high-entropy steps, especially after tool calls where uncertainty is naturally high. Its key idea is that exploration should be concentrated where the policy is unsure and where alternative continuations are likely to change downstream outcomes. This turns exploration into a targeted step-level branching problem rather than a uniform increase in randomness.

AEPO [Zhang et al., 2025a] addresses a different issue: under standard clipped objectives, high-entropy positive samples can lose their learning signal if the update is clipped too aggressively. AEPO preserves gradient flow for these useful exploratory samples and scales advantages by action entropy, thereby coupling exploration pressure to local uncertainty. In this sense, AEPO sits at the boundary between exploration design and credit design.

EPO [Xu et al., 2025] emphasizes a broader training-dynamics view. In long-horizon tasks, exploration and exploitation interact across the whole trajectory: pushing entropy at early steps can destabilize later decisions, while sparse terminal rewards may not provide enough correction. EPO therefore combines entropy regularization with entropy smoothing and adaptive schedules, framing exploration control as a stability problem rather than only a search problem.

Taken together, these methods suggest that effective exploration in agentic RL is increasingly *structured*: branch where uncertainty is high, preserve signal from informative exploratory samples, and regulate entropy over long trajectories rather than relying on uniform randomness.

The single-agent view is also no longer sufficient. FlexMARL [Jiang et al., 2026] shows that once many agents act concurrently, asynchronous rollout, team-level rewards, and hardware utilization become part of the algorithmic design space. SWEET-RL [Zhou et al., 2025], discussed above from the perspective of step-level credit, shows the complementary point: collaborative reasoning introduces an additional cross-agent credit structure on top of the within-trajectory one. Together these works suggest that multi-agent agentic RL inherits the usual long-horizon difficulties, but adds a second axis of coordination across agents that current algorithms are only beginning to handle

explicitly.

The algorithms surveyed in this section assume that trajectories can be collected efficiently enough and kept fresh enough for optimization to remain stable. The following section turns to that systems question directly.

7 Training Infrastructure for Agentic RL

7.1 Two Infrastructure Shifts

The infrastructure story of agentic RL is best understood as two consecutive lifts. The first lift goes from *ordinary LLM training and inference* to *LLM RL training and inference*: offline language-model optimization becomes an online train–generate–score–update loop. The second lift goes from *single-turn LLM RL* to *multi-turn agentic RL*: batched prompt–response rollouts become full episodes executed by an agent runtime inside a changing environment.

This distinction matters because many systems questions that look "agentic" at first actually arise one step earlier. Before dealing with tools, sandboxes, or asynchronous workers, the field first had to solve a simpler but still nontrivial problem: how to turn a standard LLM stack into a practical RL stack.

7.2 From LLM Training/Inference to LLM RL Training/Inference

Before discussing RL infrastructure, it is useful to recall the standard systems decomposition of modern LLMs. On the *training* side, practitioners typically rely on distributed training engines such as DeepSpeed, PyTorch FSDP, or Megatron-style stacks to solve data/model/pipeline parallelism, optimizer sharding, activation memory, checkpointing, and fault tolerance. On the *inference* side, they rely on specialized serving engines such as vLLM, TGI, or SGLang to maximize decoding throughput and memory efficiency through techniques such as paged KV-cache management, continuous batching, and optimized scheduling. In other words, even before RL enters the picture, the LLM ecosystem already separates the world into two relatively mature layers: a *training engine* for parameter updates and an *inference engine* for high-throughput serving.

In conventional LLM development, these two layers are largely separated. Training consumes a static dataset and optimizes one model replica under a distributed optimizer; inference serves the resulting model through a separate decoding stack. The computational pattern is relatively uniform: large dense tensor programs dominate, and the main systems questions concern parallelism, memory, checkpointing, and serving throughput.

LLM RL changes this pattern in two ways. First, *generation becomes part of training*: rollout is no longer an external evaluation phase, but an online data-collection stage inside the optimization loop. Second, the system must now coordinate multiple model roles rather than only one training model. In the canonical RLHF/RLVR setting, the actor policy, reference policy, reward model, and sometimes critic/value model must be scheduled together [Hu et al., 2024]. The resulting infrastructure problem is therefore not simply "distributed training," but *distributed co-scheduling of training and inference for multiple coupled models.*

7.3 Single-Turn LLM RL: Three Representative Abstractions

Three open frameworks illustrate the main design space for single-turn LLM RL.

TRL: trainer-centric extension of the LLM stack. TRL is the most direct lift from the Hugging Face LLM ecosystem into RL post-training [von Werra et al., 2023]. Its core abstraction is not a new distributed systems runtime, but a family of trainer classes layered on top of the familiar `transformers` training stack. This makes the transition from SFT to PPO, GRPO, RLOO, DPO, reward modeling, or KTO conceptually smooth. In infrastructure terms, TRL treats RL as an *incremental extension of the existing LLM trainer abstraction*: it is lightweight, synchronous by default, and well suited to prototyping, algorithm development, and moderate-scale post-training.

OpenRLHF: scheduling the four-model RLHF loop. OpenRLHF starts from a different systems diagnosis. Its paper emphasizes that scaling RLHF is not like scaling a single-model fine-tuning job, because the four-model loop introduces coordination and inference bottlenecks that become severe beyond small scales [Hu et al., 2024]. Its response is to explicitly combine Ray, vLLM, DeepSpeed, and Hugging Face into a framework that separates model roles and schedules them for better utilization. Compared with TRL, OpenRLHF is less about preserving the familiar trainer abstraction and more about making *multi-model RLHF scheduling* accessible and scalable.

VERL/HybridFlow: RLHF as a distributed dataflow. HybridFlow, released as VERL, pushes the abstraction one step further by viewing RLHF as a complex dataflow whose nodes are themselves distributed training or generation programs [Sheng et al., 2024]. Its key contribution is not only speed, but a cleaner decomposition of control. The paper argues that a pure single-controller design becomes inefficient once each node is itself a large distributed job, while a pure multi-controller design becomes inflexible. HybridFlow therefore introduces a hybrid control architecture together with hierarchical APIs and a 3D-HybridEngine that supports efficient actor resharding between training and generation. Conceptually, VERL/HybridFlow shows how LLM RL infrastructure matures from "a trainer with some RL extras" into *an orchestration layer for coupled distributed programs*.

Taken together, these frameworks show the first infrastructure lift clearly. TRL extends the familiar LLM trainer; OpenRLHF reorganizes RLHF around scalable multi-model scheduling; VERL/HybridFlow formalizes the whole loop as a distributed dataflow with explicit model–engine remapping. All three are primarily designed for *single-turn* RL, where a rollout is still close to "prompt in, completion out, reward back."

7.4 From Single-Turn RL to Multi-Turn Agentic RL

The second lift is more substantial. In single-turn RL, "inference" still means batched decoding from a prompt, and "rollout" is usually a short, self-contained generation. In agentic RL, inference becomes *agent execution*: the policy must read observations, emit environment-facing actions, wait for tool or environment feedback, update its context, and continue over many steps.

This shift changes the infrastructure unit from a completion to an *episode*. The system must now support persistent agent state, heterogeneous latency across steps, environment resets, tool failures, and long trajectories whose boundaries are defined by task completion rather than by a single decoding call. The training stack therefore needs more than a faster inference engine; it needs an *agent runtime* that can execute workflows, collect structured traces, and expose those traces back to RL optimization.

The core bottleneck also changes. In RLHF/RLVR, rollout cost is mostly model generation. In agentic RL, environment interaction—code execution, web browsing, database queries, API calls, GUI control—often dominates model FLOPs by orders of magnitude. This rollout–training asymmetry is the defining systems fact of agentic RL.

7.5 Training-Agent Disaggregation and Rollout-as-a-Service

AgentLightning makes this second lift explicit [Pan et al., 2025]. Its central claim is that existing approaches often couple RL too tightly to a specific agent implementation, or reduce multi-turn interaction to flat sequence concatenation and masking. AgentLightning instead advocates *complete decoupling* between the *training side* and the *agent side*. The agent executes in its native runtime; the trainer consumes a unified transition interface derived from those executions.

This design has two important consequences. First, it treats the agent runtime as a first-class systems component rather than as an inconvenient wrapper around token generation. Second, it makes agent training compatible with existing frameworks and black-box agents, because the training stack no longer assumes control over the entire execution loop. In this sense, AgentLightning is not merely a new RL algorithm paired with a new framework; it is a proposal for how the systems boundary of agentic RL should be drawn.

The natural systems pattern that follows is *Rollout-as-a-Service (RaaS)*: rollout workers execute agent–environment interaction asynchronously, while a separate trainer consumes completed trajectories from a shared data pool.

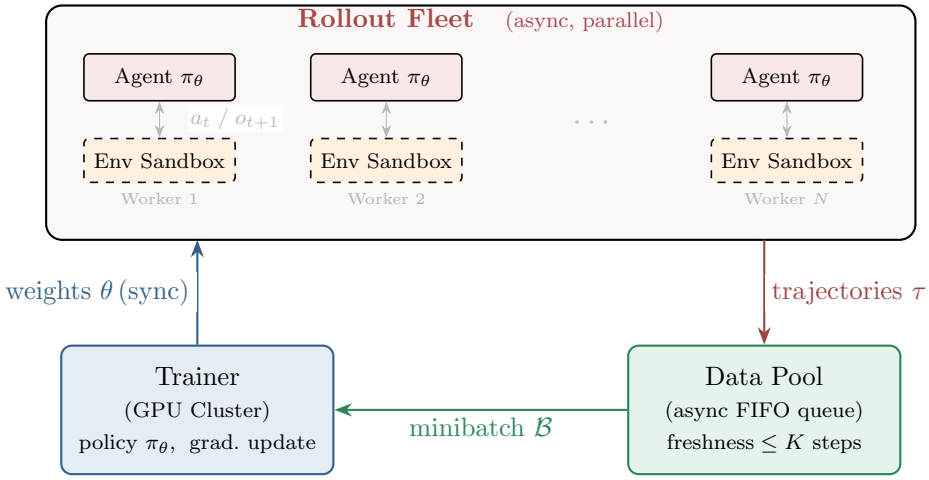

Figure 4: **Rollout-as-a-Service (RaaS) architecture for large-scale agentic RL training.** The Rollout Fleet runs N agent workers in parallel, each executing the current policy π_θ inside a sandboxed execution environment and generating complete trajectories asynchronously. Finished trajectories are deposited into a central Data Pool, which enforces a *freshness constraint*: trajectories older than K training steps are discarded to bound off-policy bias. The Trainer draws minibatches $\mathcal{B}$ from the pool for gradient updates and broadcasts updated weights back to the fleet via a weight-sync mechanism. This three-component decoupling resolves the rollout–training throughput asymmetry (Section 4): the fleet and trainer can be scaled independently, and the fleet can be expanded to handle diverse task types without modifying the training code.

RaaS is valuable because it matches the heterogeneity of agentic episodes. Some rollouts finish quickly; others stall on slow tools or long debugging sessions. Asynchronous queues therefore become necessary for throughput, while staleness control becomes necessary for training stability. MiniMax Team [2025] frame this as an *Impossible Triangle*: hardware efficiency, on-policy freshness, and agent flexibility cannot all be maximized simultaneously. Practical systems respond with bounded staleness, elastic worker pools, and queueing policies such as Windowed FIFO.

7.6 Supporting Systems: Sandboxes, Evaluation, and Reproducibility

Once rollout is agentic, several supporting systems become unavoidable. *Sandbox execution* is needed whenever the agent can run code or interact with stateful tools; throughput and fidelity must be traded off across mechanisms such as containers and lighter-weight isolation layers [Guo et al., 2025, Kimi Team, 2025]. *Online evaluation* must also be promoted from an occasional benchmark script to a standing service, since meaningful evaluation now requires rerunning agents inside environments rather than scoring static completions. Finally, *reproducibility* becomes harder because non-determinism enters through the environment itself, not only through model sampling.

These concerns are secondary in single-turn RLHF but primary in agentic RL. They are the practical consequences of moving from a pure model-centric view of RL to a system in which policy, runtime, and environment all contribute to the training signal.

7.7 Framework Comparison

Table 5 summarizes the main abstractions discussed above. The key distinction is not simply "which algorithms are implemented," but *what each framework treats as the core infrastructure object*: a trainer, a multi-model scheduler, a distributed dataflow, or a decoupled agent runtime.

Table 5: Representative open frameworks for LLM RL and agentic RL infrastructure.

Framework	Core Infrastructure Abstraction	Train/Infer Relation	Environment Assumption	Agentic Support	Main Contribution
TRL [von Werra et al., 2023]	Trainer-centric extension of the Hugging Face LLM stack	Mostly synchronous; rollout remains close to training loop	Prompt–completion style post-training	Limited	Low-barrier path from SFT to RL/post-training with broad trainer coverage
OpenRLHF [Hu et al., 2024]	Scalable scheduler for the multi-model RLHF/RLVR loop	Explicit coordination across training and inference components	Primarily single-turn RLHF/RLVR	Limited	Scales the four-model RL loop with Ray, vLLM, DeepSpeed, and HF integration
VERL / HybridFlow [Sheng et al., 2024]	Distributed RLHF dataflow with hybrid control	Flexible mapping between training and generation phases	Single-turn centered, with partial multi-turn extensibility	Partial	Hierarchical orchestration APIs and efficient actor resharding across phases
AgentLightning [Pan et al., 2025]	Decoupled training–agent architecture	Training-agent disaggregation; rollout collected externally	Native multi-turn agent execution	Native	Unifies arbitrary agent runtimes behind a common RL training interface
RAGEN [Wang et al., 2025]	Multi-turn agent RL experimentation stack	Mostly synchronous	Native multi-turn trajectories	Native	Studies multi-turn RL dynamics such as Echo Trap under GRPO-style training
Agent-R1 [Cheng et al., 2025]	Modular agent training platform	Mostly synchronous	Native multi-turn trajectories	Native	Practical environment integration and token-masked RL for real agent rollouts

The main lesson of this section is that agentic RL infrastructure is not just "larger RLHF." It is a second systems transition layered on top of the first: first from static LLM training to online LLM RL, and then from single-turn RL to full agent execution in asynchronous environments. We

now turn from this infrastructure view to the application domains and benchmarks that currently define the empirical frontier.

8 Applications and Benchmarks

8.1 Application Domain Overview

Agentic RL is now spreading into a set of application regimes that are unified less by domain semantics than by interaction structure: the agent must gather information, take environment-facing actions, and improve from delayed task-level feedback. To keep this section analytically useful, we separate two questions that are often conflated in the literature: *where* agentic RL is being applied, and *how* those applications are actually benchmarked. We therefore first review representative application families, and then summarize the benchmark landscape through the dimensions that matter most for agentic RL: environment type, reward source, horizon, and evaluation fragility. Read through the lens of the taxonomy in Section 3, the main variation across domains is not semantic topic but the combination of action space, observability, and reward structure they induce: coding and structured-data settings are attractive because rewards are relatively verifiable, whereas web, GUI, and open-ended assistant settings are more challenging precisely because observability is partial, horizons are longer, and evaluation protocols are less stable.

8.2 Agentic RAG and Deep Research

Agentic RAG extends classical retrieval-augmented generation by allowing the model to iteratively decide what to search, what to read next, when to expand evidence, and how to synthesize a final answer or report. The key distinction is that retrieval is no longer a single preprocessing step; it becomes part of the policy's interaction loop. This makes deep-research tasks a natural agentic RL setting because the quality of early retrieval decisions changes the evidence available for later reasoning and writing.

Recent search-centric systems make this progression concrete. Search-o1 [Li et al., 2025b] is an important precursor because it treats search as a first-class reasoning operation rather than as a fixed RAG preprocessor: the model learns to interleave search, reading, and reasoning over retrieved evidence. R1-Searcher [Song et al., 2025] moves further toward the RL setting by explicitly training when and how to search under outcome-based reward, showing that search invocation itself can be optimized as part of the policy rather than being hard-coded into a workflow. PaperScout [Pan et al., 2026] then pushes the design one step further by moving to a search-and-expand environment with sequence-level optimization aligned to agent–environment interaction boundaries. Taken together, these systems trace a clear line from search-augmented reasoning to search-native agentic RL.

More broadly, deep-research style evaluation increasingly emphasizes multi-hop evidence gathering, source management, and long-form synthesis rather than single-query QA alone. These tasks stress step-level credit assignment because retrieval, selection, and synthesis contribute unequally across the trajectory.

8.3 Web Navigation Agents

Web navigation provides one of the clearest fully interactive settings for agentic RL. Unlike retrieval-heavy research agents, web agents must operate inside dynamic interfaces whose content, affordances, and partial observability change after every action. The policy must decide where to click, what to

type, when to scroll, and how to recover from interface errors, often over long horizons and under delayed binary success signals.

Benchmarks such as WebArena [Zhou et al., 2024] and Mind2Web [Deng et al., 2024] expose this challenge directly. WebAgent-R1 [Wei et al., 2025] is especially relevant to the present survey because it couples multi-turn RL with explicit trajectory quality filtering and negative mining, showing that web training is limited not only by reward sparsity but also by the quality distribution of collected trajectories. This makes the web domain a natural bridge between the application discussion here and the trajectory filtering discussion in Section 5: dynamic interfaces amplify both long-horizon credit assignment and the need to curate informative partial-progress rollouts.

8.4 Coding and Software Engineering Agents

Coding remains the clearest high-value application for agentic RL because reward can often be verified automatically. In SWE-bench [Jimenez et al., 2024], the agent must inspect a repository, edit files, run tests, and iteratively debug until the issue is resolved. This gives rise to long-horizon trajectories with sparse but reliable outcome signals. HumanEval-style coding provides a lower-horizon contrast case in which execution-based verification is still available but the interaction loop is much shorter.

SWE-agent [Yang et al., 2024] established an important baseline by showing that environment interface design matters as much as policy quality: a structured agent–computer interface reduces action ambiguity and shortens the feedback loop. Subsequent agentic RL methods build on this setting because it exposes nearly all of the core technical challenges surveyed in this paper: long-horizon credit assignment, rollout cost, action irreversibility, and replay staleness.

8.5 GUI and Computer-Use Agents

Computer-use agents operate directly on rendered graphical interfaces through actions such as clicking, typing, scrolling, dragging, and window navigation. Compared with text-only tool use, this setting introduces a richer observation space and a more fragile control loop: the policy must interpret screenshots or UI trees, map them to action coordinates or interface primitives, and recover from small mistakes that can compound over long action sequences.

From the RL perspective, GUI environments are important because they push agentic training beyond symbolic tool APIs into visually grounded interaction. VeriGUI [Liu et al., 2025b] makes this shift concrete by emphasizing verifiable long-chain GUI tasks rather than short isolated clicks, while recent multimodal RL systems such as GLM-4.5V [Hong et al., 2025] show that visually grounded policy improvement is becoming practically relevant. The benchmark ecosystem here is still immature compared with coding or web navigation, but that immaturity is itself informative: GUI agent evaluation is currently bottlenecked by perception quality, action grounding, and the difficulty of specifying robust success criteria under long visual trajectories.

8.6 Game-Like and Embodied Agents

Games are a valuable agentic RL application because they provide interactive environments with clear transition dynamics, long horizons, and well-defined rewards, while remaining safer and more reproducible than many real-world deployments. In the LLM-agent setting, the most relevant game-like benchmarks are not classic Atari-style control problems, but text-rich and instruction-following environments in which the model must reason, plan, and act over many steps.

ALFWorld [Shridhar et al., 2021] is especially useful in this regard. It requires the agent to navigate, manipulate objects, and complete household tasks through a long sequence of textual

actions and observations. Such environments are important for agentic RL because they isolate sequential decision-making, exploration, and partial observability without the confounds of live web or production software systems. They therefore serve as controlled experimental arenas for studying general-purpose agent behavior and as a bridge toward more realistic embodied settings in which observation and action are no longer purely textual.

8.7 Structured Data and Analytical Agents

Structured data tasks show that agentic RL is not limited to browsing or coding. In database and table-centric environments, the agent must alternate between semantic interpretation and exact symbolic execution: understanding a user request, selecting the right schema elements, formulating intermediate transformations, issuing SQL or tool calls, and validating results against structured evidence. Benchmarks such as Spider [Yu et al., 2018] and BIRD [Li et al., 2024] are important here because they expose multi-step schema exploration and silent semantic failure modes that differ from code debugging. Systems such as MAC-SQL [Wang et al., 2024a] make this setting especially relevant to agentic RL by decomposing database reasoning into interacting subtasks whose outputs must remain execution-consistent over several rounds. Compared with web or open-ended assistant tasks, these benchmarks are often better viewed as *agentic control settings* than as fully open-world agent benchmarks, because their execution engine is relatively clean and the reward is much more mechanically specified. Their main survey value is therefore diagnostic: they provide relatively controlled evidence about semantic planning, schema exploration, and multi-turn tool mediation under exact execution feedback.

8.8 AutoResearch and Data Science Agents

AutoResearch sits one level above any single analytical benchmark: the goal is not merely to solve one forecasting, anomaly-detection, or SQL task, but to autonomously conduct a bounded research workflow over data. In this regime, the agent is expected to inspect datasets, formulate hypotheses, choose analytical tools or model families, run iterative experiments, interpret outputs, and refine the next step based on accumulated evidence. This makes the setting especially relevant to agentic RL because both the *process* and the *result* matter: a strong policy must not only reach a good final answer, but allocate computation, tool calls, and experimental budget sensibly across a long analytical trajectory.

Data-science agents such as Cast-R1 [Tao et al., 2026a] and AnomaMind [Tao et al., 2026b] are better read in this survey as early domain-specific instances of the broader AutoResearch paradigm rather than as generic structured-data benchmarks. They already exhibit the core ingredients: multi-turn tool use, iterative hypothesis refinement, and workflow-level evaluation rather than one-shot text generation. Agentic AutoML is a natural extension of the same idea. Once the action space expands from "which tool should I call next?" to "which model family, feature transformation, hyperparameter region, or validation protocol should I explore next?", the problem becomes a sequential experimental design loop rather than a static model-selection script. From the survey's perspective, the main significance of AutoResearch is therefore conceptual: it highlights a class of agentic RL problems where credit assignment must span both scientific reasoning and experimental resource allocation, and where benchmark design must measure not just correctness, but the quality and efficiency of the investigation process itself.

8.9 Collaborative Multi-Agent Systems

Multi-agent collaboration turns a long-horizon agent problem into a joint long-horizon coordination problem. Once several agents exchange messages, divide subtasks, or co-solve a task, communication itself becomes part of the action space and reward must be attributed across interacting trajectories rather than within one trajectory alone.

SWEET-RL [Zhou et al., 2025] and FlexMARL [Jiang et al., 2026] represent two complementary points in this emerging regime. SWEET-RL studies collaborative reasoning tasks in which the agent must improve its interaction with a partner over multiple rounds, using step-level rewards derived from training-time information to provide denser supervision. FlexMARL addresses the systems side of the same problem by co-designing rollout and training for many-agent settings, emphasizing asynchronous collection, efficient utilization, and shared-reward optimization across teams of agents. The broader point is that multi-agent agentic RL is already transitioning from an abstract future direction to a concrete experimental setting with dedicated benchmarks, algorithms, and systems designs.

8.10 Open-Ended Assistant Agents

Open-ended assistant tasks matter because they sit closest to the product-facing ambition of agentic RL: the agent must combine retrieval, browsing, tool use, code execution, and long-form synthesis within one episode rather than inside a narrow vertical. GAIA [Mialon et al., 2023] is the clearest benchmark exemplar here because it couples heterogeneous tools with end-to-end task completion under long horizons. Systems such as ProRL Agent [Pang et al., 2025] show that this regime is not merely an evaluation curiosity; it already pressures the infrastructure choices discussed in Section 7, especially rollout–training decoupling and recent-trajectory reuse for very long episodes.

8.11 Benchmark and Evaluation Landscape

Application domains and benchmark suites are not interchangeable. A domain paragraph explains why a task family matters; a benchmark analysis must additionally explain what is actually measured, under what environment assumptions, and how portable the resulting numbers are across papers. In agentic RL this distinction is especially important because benchmark names often hide major protocol differences: static versus live environments, binary success versus action matching, different trajectory budgets, and different levels of scaffold support.

The resulting benchmark ecosystem is heterogeneous in a structurally important way. SWE-bench is repository-grounded and verifier-backed; WebArena is a controlled but fully interactive browser simulator; Mind2Web is closer to an action-supervision corpus over real websites; Spider and BIRD are execution-grounded structured reasoning environments; and GAIA is an open-ended tool suite whose reported performance is highly scaffold-sensitive. These settings all matter, but they do not validate the same claims. A gain on a verifier-backed structured task is strong evidence about reward utilization and credit assignment under clean feedback; a gain on web or open-ended assistant tasks is more informative about robustness, long-horizon planning, and systems design under brittle evaluation.

Table 6 summarizes the current benchmark landscape through the lens most relevant to agentic RL: the environment the agent interacts with, the source of reward, the typical horizon, and the main technical stress point. To avoid sliding back into method cataloging, the table pairs each regime with the *kind* of agentic RL study it typically supports rather than reproducing a system-by-system leaderboard.

Table 6: Representative application regimes, benchmark environments, and the benchmark interpretations they most directly support.

Application Regime	Representative Benchmarks / Environments	Typical RL Study Framing	Reward Signal	Typical Horizon	Main Stress Point
Deep Research / Retrieval	Search-o1-style environments; PaperScout tasks	RL for search invocation, evidence expansion, and long-form synthesis	Search success; citation quality; task completion	M–L	Multi-hop retrieval, query refinement, source selection, and long-form synthesis
Web Navigation	WebArena; Mind2Web	Outcome-based policy improvement under dynamic interfaces and partial-progress rollouts	Task success; action matching; partial progress	L	Dynamic interfaces, DOM partial observability, and recovery from compounding errors
Coding / Software Engineering	SWE-bench; HumanEval-style coding	Verifier-backed long-horizon code repair and debugging	Executable tests; patch success	M–L	Long-horizon debugging, file editing, and sparse but reliable outcome rewards
Computer Use (GUI)	VeriGUI and related long-chain GUI tasks	Visually grounded long-chain control with fragile action grounding	Visual task completion	L	Visual grounding, long action chains, partial observability, and fragile control
Game-Like / Embodied Control	ALFWorld and related interactive environments	Controlled sequential decision-making in reproducible interactive worlds	Environment-defined task reward	M–L	Exploration, planning, partial observability, and recovery from compounding errors
Structured Data / Database Reasoning	Spider; BIRD	Execution-grounded semantic control over clean symbolic backends	Execution match	M	Schema navigation, exact computation, and multi-turn control under relatively clean verification
AutoResearch / Data Science	Cast-R1; AnomaMind; GAIA-style open analysis tasks	Workflow-level hypothesis refinement, tool/model selection, and experimental budget allocation	Workflow-level correctness; experiment quality; end-task improvement	M–L	Iterative hypothesis testing, tool/model selection, and experimental budget allocation over long analytical loops
Multi-Agent Collaboration	SWEET-RL collaborative tasks; FlexMARL suites	Shared-reward coordination over communicating agents	Shared team reward; collaborator success	M–L	Inter-agent credit assignment, communication, and joint non-stationarity
Open-Ended Assistants	GAIA and related multi-tool assistant settings	Heterogeneous tool orchestration under long horizons and scaffold-sensitive evaluation	End-to-end completion with mixed process signals	XL	Heterogeneous tool orchestration, long horizons, and high scaffold sensitivity

Interpretation note. These rows should not be read as equally difficult or directly comparable. For example, *SWE-bench* uses repository-level executable verification, *Mind2Web* emphasizes action supervision over a broader website distribution, and *GAIA* is highly sensitive to tool configuration and action budget. The table is intended to expose structural differences in environment and evaluation protocol, not to imply a single leaderboard across regimes.

Table 7 makes these protocol differences explicit in a compact form. The table is intentionally selective rather than exhaustive: it covers a small set of canonical benchmarks whose protocol assumptions are relatively legible, and should be read as a comparison aid rather than as a complete inventory of GUI, multi-agent, or data-science evaluation settings. The point is not to exhaustively describe each benchmark, but to expose the minimum dimensions a reader needs before treating reported numbers as comparable.

Table 7: A compact protocol taxonomy for representative agentic RL benchmarks.

Benchmark	**Environment Regime**	**Primary Supervision**	**Scaffold Sensitivity**	**Best Read As**
SWE-bench	Static repository snapshot	Executable binary verification	Medium: depends on harness, edit tools, and trajectory budget	Verifier-backed agentic control benchmark for long-horizon software tasks
WebArena	Interactive simulator	End-task success in a controlled browser world	High: depends on browser scaffold and action budget	Interactive planning benchmark with relatively controlled environment dynamics
Mind2Web	Real-website workflow corpus	Action supervision / trajectory matching	High: depends on action formulation and website variation	Web generalization benchmark, less clean as a pure RL outcome benchmark
Spider / BIRD	Structured execution environment	Execution-match accuracy	Medium: tool use matters, but verification is relatively clean	Structured reasoning control benchmark with stronger symbolic grounding
GAIA	Open-ended multi-tool environment	End-to-end task completion with mixed process constraints	Very high: tool set, prompts, and budget strongly affect results	Frontier assistant benchmark for heterogeneous, scaffold-sensitive agent behavior

We intentionally do not reproduce a second cross-system method survey here. Those design differences are analyzed in Sections 6 and 7; within the present section, the main question is what a benchmark result should be taken to mean. Read that way, coding and structured-execution gains are strongest as evidence of reward utilization under relatively clean verification, web and GUI gains are strongest as evidence of robustness under partial observability and brittle interaction loops, and GAIA-style assistant gains are strongest as evidence of workflow orchestration under scaffold-sensitive evaluation.

Three patterns emerge from this application and benchmark view. First, the strongest current agentic RL results still tend to appear where rewards are at least partially verifiable, especially in coding, structured-data execution, and some analytical workflows. Second, not all verifiable tasks are equally agentic: structured execution benchmarks are valuable control settings, but the more frontier-style evidence for agentic RL still comes from environments in which actions substantially reshape future observations, such as web navigation, GUI control, and open-ended assistants. Third, these applications collectively show that agentic RL is increasingly defined by *workflow control* over external environments rather than by any single domain; in the multi-agent case, that workflow control must extend across communicating policies rather than within one policy alone.

For this reason, benchmark numbers in agentic RL should be interpreted more cautiously than static NLP leaderboards. A result on SWE-bench depends on the split and harness configuration; a result on WebArena or Mind2Web depends on simulator version, website state, and success metric; a result on GAIA depends on the available tool set and maximum action budget. The practical lesson is that benchmark names alone are not enough for comparison. Reproducible reporting requires the environment snapshot, scaffold, tool configuration, trajectory budget, and evaluation protocol to be treated as first-class experimental variables rather than appendices.

9 Open Challenges and Future Directions

The preceding sections surveyed the current state of agentic RL across algorithms, trajectory representations, and training infrastructure. Here we shift from *current bottlenecks* to *frontier questions*: problems that remain unresolved despite existing mitigations and for which the field still lacks a stable methodology, convincing abstraction, or reliable evaluation protocol.

Rather than treating the frontier as nine unrelated items, it is more useful to group the open problems into four clusters that follow the training pipeline: *learning signals and supervision*, *initialization and sample efficiency*, *evaluation and training systems*, and *theory and scaling*. This grouping keeps the original problems intact, but makes their dependencies clearer.

9.1 Learning Signals and Supervision

Large-Scale Exploration. The first cluster begins with exploration at trajectory scale. In agentic RL, the object of search is no longer a set of alternative final responses, but a distribution over multi-step action trajectories whose intermediate decisions recursively shape subsequent observations and future opportunities for reward. As episode horizon increases, successful trajectories become exponentially sparse, while naive stochastic sampling tends to concentrate around high-probability behavioral modes of the current policy rather than discovering qualitatively new strategies. Existing approaches, including intrinsic-motivation objectives [Bellemare et al., 2016, Pathak et al., 2017], structured search methods [Hao et al., 2023], entropy-based regularization [Xu et al., 2025], and population-based diversity mechanisms [Mouret and Clune, 2015], each address parts of this problem but have not yet yielded a generally effective exploration framework for long-horizon language agents. The central open question is therefore not merely how to increase randomness in policy rollouts, but how to define behaviorally meaningful novelty, allocate exploration budget across long trajectories, and characterize adequate coverage in compositional language-action spaces.

Reward Design Beyond Verifiable Outcomes. Exploration immediately meets the second problem: *automatic* reward specification once outcomes are no longer fully verifiable. The strongest current results still concentrate in settings where code passes tests, proofs can be checked, or answers

match an external verifier, because these tasks provide scalable and relatively low-noise reward signals. Many consequential future uses of agentic systems—scientific discovery, project coordination, open-ended research assistance, and strategic planning—do not reduce to a binary verifier. The open problem is therefore not simply "how to learn a reward model," but how to specify *interactive progress* in environments where success is delayed, partial, and multi-objective. Agentic tasks often conflate outcome correctness, interaction efficiency, and trajectory quality. Existing approaches, including PRMs [Lightman et al., 2023] and model-based evaluators, illuminate parts of this problem but do not resolve it; the field still needs reward formulations that capture partial progress without collapsing into brittle proxy objectives.

Human Feedback in the Loop. Once automatic rewards become too coarse, the next question is how to incorporate human feedback once training moves beyond static output comparisons. RLHF modeled human supervision mainly as preferences over final responses [Christiano et al., 2017, Ouyang et al., 2022], but agentic RL requires a richer treatment: humans may judge full trajectories, correct single steps, approve actions before execution, or provide free-form critiques after failure. These signals differ in timing, cost, and bias, and they do not map cleanly onto a single reward model. The open challenge is therefore to treat human feedback as structured information inside a sequential decision process rather than as a static label. More general views of learning from human feedback [Azar et al., 2024] point in this direction, but scalable algorithms remain limited; a particularly important direction is *selective oversight*, namely learning when human input is worth requesting, how to combine sparse human supervision with automatic rewards, and how to do so without making the training objective internally inconsistent across turns.

9.2 Initialization and Sample Efficiency

Cold Start and Hybrid Training with SFT/SSL. The second cluster concerns how promising trajectories are made reachable in the first place. Agentic RL rarely starts from scratch; in practice it sits on top of pretraining and usually some amount of supervised fine-tuning, which turns cold start into a recipe-design problem rather than a single algorithmic one. Weak initial policies may never reach rewarding trajectories, yet demonstrations alone often fail to capture the recovery loops, branching decisions, and tool-use detours that matter most in multi-turn settings. The open question is therefore not simply whether agentic systems need warm starts, but what *training schedule* should connect SFT, self-supervision, and RL. Recent systems already explore different recipes: Kimi k1.5 couples RL with phase-transfer and curriculum-style scheduling [Kimi Team, 2025], Absolute Zero pushes toward self-generated tasks and reinforced self-play [Zhao et al., 2025], and MiniMax-M1 mixes rollout filtering, staged transfer, and long-horizon optimization [MiniMax Team, 2025]. Yet the field still lacks consensus on whether these components should be organized in clean stages or interleaved cycles, and on which warm-start source is most effective: expert traces, self-play, archived user data, or lightweight synthetic tasks.

Sample Efficiency Without Full World Models. Initialization alone is insufficient if each trial remains too expensive. Current agentic RL systems remain costly because long-horizon rollouts are expensive, sparse-reward tasks provide little usable signal per episode, and environment interaction often dominates the total training budget. One possible response is *world modeling*, but it should not be treated as the only route to efficiency. A more complete view contains three main paths: trajectory reuse through replay, retrieval, and relabeling; selective model-based augmentation through short-horizon simulated branches; and full environment modeling with synthetic rollouts. Each path has its own bias–variance tradeoff: replay creates staleness and off-policy bias, selective

simulation depends on where branching is worthwhile, and full simulation is limited by model inaccuracy and compounding error [Janner et al., 2019]. The central open question is therefore not simply whether LLMs can act as world models, but how reuse, selective simulation, and environment modeling should be combined to reduce real-environment interaction cost without corrupting the training signal.

9.3 Evaluation and Training Systems

Evaluation Beyond Static Snapshot Benchmarks. The third cluster concerns whether the field can make cumulative empirical claims at all. Benchmarks in agentic RL are improving, but the standards for interpreting benchmark numbers remain weak. Four gaps matter most: static snapshot bias, limited measurement of transfer beyond in-distribution tasks, poor reproducibility under environment stochasticity, and the ambiguity of ground truth in open-ended tasks. What is missing is not just more benchmarks, but stronger evaluation protocols. At minimum, agentic RL papers should report environment snapshot or version, tool configuration, maximum trajectory budget, number of evaluation runs, and uncertainty estimates such as confidence intervals. For tasks where binary success is too coarse, reports should also include partial-progress or rubric-based metrics. Until these distinctions are standardized, cross-paper comparisons will remain informative but not cumulative.

Training Engines and Research Accessibility. Evaluation standards are difficult to stabilize if training pipelines remain hard to reproduce. Agentic RL systems are often built as tightly coupled full-stack pipelines, making them powerful but difficult to modify, while lighter frameworks are easier to adopt but often too thin for serious multi-turn training. Frameworks such as HybridFlow/VERL [Sheng et al., 2024], OpenRLHF [Hu et al., 2024], AgentLightning [Pan et al., 2025], ProRL Agent [Pang et al., 2025], and VerlTool [Guo et al., 2025] help narrow this gap, but no dominant abstraction has emerged for integrating environment interaction, trajectory storage, credit assignment, and policy optimization. The unresolved issue is not merely engineering maturity, but interface design: should the core training object be a flat message trace or an explicit transition tuple, and where should the systems boundary sit between agent runtime, environment wrapper, and trainer? A mature training engine should therefore be modular enough that new algorithms do not require rewriting the whole stack, yet opinionated enough to make results portable rather than pipeline-specific.

9.4 Theory and Scaling

Theory for Structured, Asynchronous, Multi-Turn Optimization. The fourth cluster addresses whether current empirical practice can be explained and predicted rather than merely reported. Theoretical discussion is most useful when it stays close to the regime that current systems actually inhabit: structured actions, partial observability, asynchronous rollout, replay staleness, and long-horizon delayed reward. The central unanswered question is *stability*: PPO-, GRPO-, and related methods combine clipping, normalization, KL anchoring, token masking, and trajectory reuse in ways that standard policy-gradient analyses do not cover [Sutton et al., 1999]. This gap is especially visible for the branch-based and replay-heavy methods surveyed earlier, such as GiGPO, Tree-GRPO, RAPO, and AgentHER, whose empirical gains still lack a clear account of when branching, relabeling, or replay improve optimization rather than introduce new bias. A practical theory would explain when these interactions create collapse, divergence, or biased updates,

and how representation choices such as token-level versus step-level optimization alter those failure modes.

Scaling Laws for Agentic RL. Theory in turn should support a more principled account of scaling. Scaling laws organize modern LLM development [Kaplan et al., 2020], but their agentic counterpart remains largely unknown. Agentic RL introduces variables beyond model size and update compute, including rollout cost, horizon length, environment fidelity, tool diversity, memory budget, and available oversight. These make scaling substantially harder to measure. The issue is already visible in systems such as Kimi k1.5 and MiniMax-M1, where performance depends not only on model capacity but also on partial-rollout scheduling, long-to-short transfer, curriculum design, and the ratio between rollout generation and policy updates [Kimi Team, 2025, MiniMax Team, 2025]. Performance is usually reported as thresholded task success rather than a smooth loss, and rollout compute may matter as much as optimization compute. The practical open question is whether agentic RL admits a compute-optimal frontier that jointly allocates model size, rollout budget, and environment interaction cost across different environment classes.

Table 8 summarizes these frontier questions by cluster, emphasizing not only what remains open, but also what kind of empirical evidence would count as concrete progress.

Table 8: Open problems in agentic RL and what would count as concrete progress.

Cluster	Problem	Why current methods are insufficient	Promising directions	What would count as progress
Learning signals and supervision	Large-scale exploration	Local branching and entropy heuristics improve nearby search but do not reliably discover qualitatively new strategies.	Population diversity, hierarchical subgoals, imagination-guided search, semantics-aware exploration metrics.	Higher success on long-horizon tasks under fixed rollout budget, together with evidence of genuinely diverse strategy families rather than surface variation.
Learning signals and supervision	Reward design	Automatic outcome rewards miss partial progress, efficiency, and process quality; dense shaping is easy to game.	PRMs, rubric-based evaluators, multi-objective reward design, learned subgoals with alignment checks.	Better correlation with expert judgment, reduced reward hacking, and improved learning on open-ended tasks without collapsing to proxy objectives.
Learning signals and supervision	Human feedback in the loop	Final-output preferences alone cannot capture trajectory-level judgments, action approvals, or sparse critiques in multi-turn environments.	Selective oversight, trajectory-level annotation, step correction, and hybrid human-plus-automatic feedback schemes.	Human supervision can be added where it is most informative without overwhelming cost or creating contradictory training signals.
Initialization and sample efficiency	Cold start and hybrid training	Weak initial policies may never reach rewarding trajectories, while demonstrations alone miss recovery loops and branching behavior.	Staged or interleaved SFT/SSL/RL schedules, self-play warm starts, archived user traces, and lightweight synthetic tasks.	Reliable training recipes that improve long-horizon performance without overconstraining exploration from the outset across different environments and warm-start sources.
Initialization and sample efficiency	Sample efficiency	Real-environment rollouts remain too expensive; full world models are not yet reliable enough to replace them.	Replay valuation, hindsight relabeling, retrieval-augmented reuse, short-horizon selective simulation.	Comparable or better policy quality at substantially lower real-environment interaction cost.
Evaluation and training systems	Evaluation	Static snapshots, single-run reporting, and binary metrics make cross-paper comparison fragile.	Versioned environments, OOD splits, variance reporting, partial-progress metrics, controlled scaffold disclosure.	Reproducible multi-run benchmarks with confidence intervals and meaningful transfer evaluation across changed environments.
Evaluation and training systems	Training engines	Frontier systems are often too coupled to extend, while lightweight frameworks are often too thin for serious agent training, leaving core interfaces underspecified.	Modular training engines, unified trajectory interfaces, decoupled rollout/training stacks, and reusable environment adapters.	New algorithms can be implemented and reproduced without bespoke end-to-end systems work for each project or lab-specific runtime assumptions.
Theory and scaling	Theory	Practice depends on clipping, masking, replay, and asynchronous rollout choices that still lack explanatory theory.	Stability analysis for group-relative methods, bias analysis for truncation and replay, and theory for token-level versus step-level optimization.	Predictive accounts of failure modes such as collapse, divergence, and replay-induced bias in long-horizon training.
Theory and scaling	Scaling laws	Performance depends on coupled model, rollout, and environment budgets rather than a single compute axis, and current metrics are too thresholded to expose smooth trends.	Efficiency-adjusted metrics, environment-class-specific scaling studies, and compute-allocation analyses spanning rollout and optimization.	Reliable scaling curves that indicate how budget should be divided among model size, rollout generation, and environment interaction.

These open challenges still form a dependency chain, but the chain is easiest to read at the level of clusters. Learning signals determine whether useful behavior can be discovered at all; initialization and sample-efficiency methods determine whether those behaviors are reachable within practical budgets; evaluation and training systems determine whether reported gains are reproducible and

cumulative; and only then can theory and scaling studies become explanatory rather than anecdotal. Taken together, they mark the transition that agentic RL must make from promising systems demonstrations to a mature science of multi-turn agent optimization.

10 Conclusion

This survey has traced the evolution of reinforcement learning for large language models through three generations, unified by a single structural axis: the transition from *single-turn feedback* to *multi-turn interaction*. RLHF and RLVR both treat training as a bandit-style problem in which one response receives one scalar judgment, whether from a human or a verifier. Agentic RL changes the problem class itself. Once an LLM acts across multiple turns in a stateful environment, learning becomes a sequential decision problem with partial observability, delayed consequences, and environment-dependent feedback. This shift is the central lens through which we have organized the survey: it explains why algorithms designed for single-response optimization transfer only partially, why trajectory representation and credit assignment become first-class design choices, and why training infrastructure must now absorb the cost and brittleness of real environment interaction.

Across this landscape, several conclusions recur. Agentic RL is not defined by any single application domain, but by workflows in which actions alter future observations and thus reshape the rest of the trajectory. This is why coding, web navigation, deep research, structured-data analytics, computer use, and collaborative multi-agent settings all expose related challenges despite their different surface semantics. The survey has argued that the core technical bottlenecks remain long-horizon credit assignment, trajectory filtering and reuse, evaluation instability across heterogeneous environments, and rollout–training asymmetry at the systems level. Current methods already provide useful partial solutions, but the field does not yet have stable answers to how rewards should be attributed over long interaction traces, how agent trajectories should be represented for learning, or how results should be compared across scaffolds, environments, and tool configurations.

Taken together, the literature suggests that agentic RL is becoming a distinct technical regime rather than a simple extension of earlier LLM alignment pipelines. Its progress will likely depend not only on better policy optimization, but also on more reproducible benchmark infrastructure, clearer environment specifications, stronger support for memory and multi-agent interaction, and tighter co-design between learning algorithms and distributed execution systems. The evidence reviewed here therefore points to a field that is already rich enough to merit systematic synthesis, but still early enough that many of its conceptual boundaries, empirical standards, and systems abstractions remain unsettled. Clarifying those boundaries is precisely what will determine whether agentic RL matures into a cumulative and dependable discipline for training interactive language agents.

References

A. Ahmadian, C. Cremer, M. Gallé, M. Fadaee, J. Kreutzer, O. Pietquin, A. Üstün, and S. Hooker. Back to basics: Revisiting REINFORCE style optimization for learning from human feedback in LLMs. *arXiv preprint arXiv:2402.14740*, 2024.

M. G. Azar, Z. D. Guo, B. Piot, R. Munos, M. Rowland, M. Valko, and D. Calandriello. A general theoretical paradigm to understand learning from human feedback. *arXiv preprint arXiv:2310.12036*, 2024.

Y. Bai, S. Kadavath, S. Kundu, A. Askell, J. Kernion, A. Jones, A. Chen, A. Goldie, A. Mirhoseini, C. McKinnon, et al. Constitutional ai: Harmlessness from ai feedback. *arXiv preprint arXiv:2212.08073*, 2022.

M. Bellemare, S. Srinivasan, G. Ostrovski, T. Schaul, D. Saxton, and R. Munos. Unifying count-based exploration and intrinsic motivation. In *Advances in Neural Information Processing Systems*, volume 29, 2016.

S. Bensal, U. Jamil, C. Bryant, M. Russak, K. Kamble, D. Mozolevskyi, M. Ali, and W. Al-Shikh. Reflect, retry, reward: Self-improving LLMs via reinforcement learning. *arXiv preprint arXiv:2505.24726*, 2025. URL `https://arxiv.org/abs/2505.24726`.

S. Casper, X. Davies, C. Shi, T. K. Gilbert, J. Scheurer, J. Rando, R. Freedman, T. Korbak, D. Lindner, P. Freire, et al. Open problems and fundamental limitations of reinforcement learning from human feedback. *arXiv preprint arXiv:2307.15217*, 2023.

Z. Chen et al. Tree search for LLM agent reinforcement learning. 2025. arXiv:2509.21240.

M. Cheng et al. Agent-R1: Training powerful LLM agents with end-to-end reinforcement learning. 2025. arXiv:2511.14460.

P. F. Christiano, J. Leike, T. Brown, M. Martic, S. Legg, and D. Amodei. Deep reinforcement learning from human preferences. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems*, volume 35, pages 16344–16359, 2022.

DeepSeek-AI. Deepseek-r1: Incentivizing reasoning capability in LLMs via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.

X. Deng, Y. Gu, B. Zheng, S. Chen, S. Stevens, B. Wang, H. Sun, and Y. Su. Mind2Web: Towards a generalist agent for the web. *Advances in Neural Information Processing Systems*, 36, 2024.

K. Ethayarajh, W. Xu, N. Muennighoff, D. Jurafsky, and D. Kiela. KTO: Model alignment as prospect theoretic optimization. *arXiv preprint arXiv:2402.01306*, 2024.

X. Guan, L. L. Zhang, Y. Liu, N. Shang, Y. Sun, Y. Zhu, F. Yang, and M. Yang. rStar-math: Small LLMs can master math reasoning with self-evolved deep thinking. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 20640–20661. PMLR, 2025. URL `https://proceedings.mlr.press/v267/guan25f.html`.

H. Guo et al. VerlTool: Holistic agent reinforcement learning with tool use. 2025. arXiv:2509.01055.

S. Hao, Y. Gu, H. Ma, J. J. Hong, Z. Wang, D. Z. Wang, and Z. Hu. Reasoning with language model is planning with world model. *arXiv preprint arXiv:2305.14992*, 2023.

W. Hong, W. Yu, X. Gu, G. Wang, G. Gan, H. Tang, J. Cheng, J. Qi, J. Ji, L. Pan, et al. GLM-4.5v and GLM-4.1v-thinking: Towards versatile multimodal reasoning with scalable reinforcement learning. *arXiv preprint arXiv:2507.01006*, 2025.

J. Hu et al. OpenRLHF: An easy-to-use, scalable and high-performance RLHF framework. `https://github.com/OpenRLHF/OpenRLHF`, 2024.

M. Janner, J. Fu, M. Zhang, and S. Levine. When to trust your model: Model-based policy optimization. In *Advances in Neural Information Processing Systems*, volume 32, 2019.

Z. Jiang et al. Rollout-training co-design for efficient LLM-based multi-agent reinforcement learning. 2026. arXiv:2602.09578.

C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? *arXiv preprint arXiv:2310.06770*, 2024.

J. Kang, M. Ji, Z. Zhao, and T. Bai. Memory OS of AI agent. *arXiv preprint arXiv:2506.06326*, 2025. URL `https://arxiv.org/abs/2506.06326`.

J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, and D. Amodei. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.

A. Kazemnejad, M. Aghajohari, E. Portelance, A. Sordoni, S. Reddy, A. Courville, and N. Le Roux. VinePPO: Refining credit assignment in RL training of LLMs. *arXiv preprint arXiv:2410.01679*, 2024.

Kimi Team. Kimi k1.5: Scaling reinforcement learning with LLMs. *arXiv preprint arXiv:2501.12599*, 2025.

H. Lee, S. Phatale, H. Mansoor, K. Lu, T. Mesnard, C. Bishop, V. Carbune, and A. Rastogi. Rlaif: Scaling reinforcement learning from human feedback with ai feedback. *arXiv preprint arXiv:2309.00267*, 2023.

J. Li, B. Hui, G. Qu, J. Yang, B. Li, B. Li, B. Wang, B. Qin, R. Geng, N. Huo, et al. BIRD: A big bench for large-scale database grounded text-to-SQL evaluation. In *Advances in Neural Information Processing Systems*, volume 36, 2024.

M. Li et al. SkyRL-agent: Efficient reinforcement learning for multi-turn LLM agents. 2025a. arXiv:2511.16108.

X. Li et al. Search-o1: Agentic search-enhanced large reasoning models. 2025b. arXiv:2501.05366.

Y. Li, X. Shen, X. Yao, X. Ding, Y. Miao, K. Ramayya, and R. Padman. Beyond single-turn: A survey on multi-turn interactions with large language models. *arXiv preprint arXiv:2504.04717*, 2026.

H. Lightman, V. Kosaraju, Y. Burda, H. Edwards, B. Baker, T. Lee, J. Leike, J. Schulman, I. Sutskever, and K. Cobbe. Let's verify step by step. *arXiv preprint arXiv:2305.20050*, 2023.

M. Lin, Z. Wu, Z. Xu, H. Liu, X. Tang, Q. He, C. Aggarwal, H. Liu, X. Zhang, and S. Wang. A comprehensive survey on reinforcement learning-based agentic search: Foundations, roles, optimizations, evaluations, and applications. *arXiv preprint arXiv:2510.16724*, 2025.

K. Liu, D. Yang, Z. Qian, W. Yin, Y. Wang, H. Li, J. Liu, P. Zhai, Y. Liu, and L. Zhang. Reinforcement learning meets large language models: A survey of advancements and applications across the LLM lifecycle. *arXiv preprint arXiv:2509.16679*, 2025a.

S. Liu, M. Liu, H. Zhou, Z. Cui, Y. Zhou, Y. Zhou, W. Fan, G. Zhang, J. Shi, W. Xuan, et al. VeriGUI: Verifiable long-chain GUI dataset. *arXiv preprint arXiv:2508.04026*, 2025b.

K. Lu et al. A practitioner's guide to continual multimodal pretraining. 2024. arXiv:2510.01132.

J. Luo et al. Group-in-group policy optimization for LLM agent training. 2025. arXiv:2505.10978.

Y. Meng, M. Xia, and D. Chen. Simpo: Simple preference optimization with a reference-free reward. *arXiv preprint arXiv:2405.14734*, 2024.

G. Mialon, C. Fourrier, C. Swift, T. Wolf, Y. LeCun, and T. Scialom. GAIA: A benchmark for general AI assistants. *arXiv preprint arXiv:2311.12983*, 2023.

MiniMax Team. MiniMax-M1: Scaling test-time compute efficiently with lightning attention. *arXiv preprint arXiv:2506.13585*, 2025.

J.-B. Mouret and J. Clune. Illuminating search spaces by mapping elites. *arXiv preprint arXiv:1504.04909*, 2015.

L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, et al. Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems*, 35:27730–27744, 2022.

T. Pan, J. Ouyang, M. Cheng, Q. Li, Z. Liu, D. Wang, M. Pan, S. Yu, and Q. Liu. PaperScout: An autonomous agent for academic paper search with process-aware sequence-level policy optimization. 2026. arXiv:2601.10029.

Y. Pan et al. AgentLightning: Train any AI agents with reinforcement learning. 2025. arXiv:2508.03680.

J. Pang et al. ProRL agent: Rollout-as-a-service for scalable agentic reinforcement learning. 2025. arXiv:2603.18815.

D. Pathak, P. Agrawal, A. A. Efros, and T. Darrell. Curiosity-driven exploration by self-supervised prediction. In *International Conference on Machine Learning*, pages 2778–2787, 2017.

A. Plaat, M. van Duijn, N. van Stein, M. Preuss, P. van der Putten, and K. J. Batenburg. Agentic large language models, a survey. *arXiv preprint arXiv:2503.23037*, 2025.

H. Qiu et al. Scaling reinforcement learning for LLM agents with multi-turn, multi-task interactions. 2025. arXiv:2510.04206.

R. Rafailov, A. Sharma, E. Mitchell, C. D. Manning, S. Ermon, and C. Finn. Direct preference optimization: Your language model is secretly a reward model. *Advances in Neural Information Processing Systems*, 36, 2023.

J. Sang, J. Xiao, J. Han, J. Chen, X. Chen, S. Wei, Y. Sun, and Y. Wang. Beyond pipelines: A survey of the paradigm shift toward model-native agentic AI. *arXiv preprint arXiv:2510.16720*, 2025.

J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal policy optimization algorithms. In *arXiv preprint arXiv:1707.06347*, 2017.

Z. Shao, P. Wang, Q. Zhu, R. Xu, J. Song, M. Zhang, Y. Li, Y. Wu, and D. Guo. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

G. Sheng et al. HybridFlow: A flexible and efficient RLHF framework. *arXiv preprint arXiv:2409.19256*, 2024.

M. Shridhar, X. Yuan, M.-A. Côté, Y. Bisk, A. Trischler, and M. Shapiro. ALFWorld: Aligning text and embodied environments for interactive learning. In *International Conference on Learning Representations*, 2021.

H. Song et al. R1-Searcher: Incentivizing the search capability in LLMs via reinforcement learning. 2025. arXiv:2503.05592.

S. S. Srivastava and V. Aggarwal. A technical survey of reinforcement learning techniques for large language models. *arXiv preprint arXiv:2507.04136*, 2025.

R. S. Sutton, D. McAllester, S. Singh, and Y. Mansour. Policy gradient methods for reinforcement learning with function approximation. In *Advances in Neural Information Processing Systems*, volume 12, 1999.

X. Tao, M. Cheng, C. Jiang, T. Gao, H. Zhang, and Y. Liu. Cast-R1: Learning tool-augmented sequential decision policies for time series forecasting. 2026a. arXiv:2602.13802.

X. Tao, Y. Wu, M. Cheng, Z. Guo, and T. Gao. AnomaMind: Agentic time series anomaly detection with tool-augmented reasoning. 2026b. arXiv:2602.13807.

L. von Werra et al. TRL: Transformer reinforcement learning. `https://github.com/huggingface/trl`, 2023.

B. Wang, C. Ren, J. Yang, X. Liang, J. Bai, Q.-W. Zhang, Z. Yan, and Z. Li. MAC-SQL: A multi-agent collaborative framework for text-to-SQL. *arXiv preprint arXiv:2312.11242*, 2024a.

L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, Z. Chen, J. Tang, X. Chen, Y. Lin, et al. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18 (6):186345, 2024b.

L. Wang et al. TAMTRL: Teacher-aligned reward reshaping for multi-turn reinforcement learning in long-context compression. 2026. arXiv:2603.21663.

Z. Wang et al. RAGEN: Understanding self-evolution in LLM agents via multi-turn reinforcement learning. 2025. arXiv:2504.20073.

Z. Wei et al. WebAgent-R1: Training web agents via end-to-end multi-turn reinforcement learning. 2025. arXiv:2505.16421.

Z. Xi, W. Chen, X. Guo, W. He, Y. Ding, B. Hong, M. Zhang, J. Wang, S. Jin, E. Zhou, et al. The rise and potential of large language model based agents: A survey. *arXiv preprint arXiv:2309.07864*, 2023.

P. Xia, J. Chen, H. Wang, J. Liu, K. Zeng, Y. Wang, S. Han, Y. Zhou, X. Zhao, H. Chen, Z. Zheng, C. Xie, and H. Yao. SkillRL: Evolving agents via recursive skill-augmented reinforcement learning. *arXiv preprint arXiv:2602.08234*, 2026. URL `https://arxiv.org/abs/2602.08234`.

W. Xu, W. Zhao, Z. Wang, Y.-J. Li, C. Jin, M. Jin, K. Mei, K. Wan, and D. N. Metaxas. EPO: Entropy-regularized policy optimization for LLM agents reinforcement learning. *arXiv preprint arXiv:2509.22576*, 2025. URL `https://arxiv.org/abs/2509.22576`.

J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems*, 2024.

S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023.

Q. Yu, Z. Zhang, R. Zhu, Y. Yuan, X. Dong, et al. DAPO: An open-source LLM reinforcement learning system at scale. *arXiv preprint arXiv:2503.14476*, 2025a.

T. Yu, R. Zhang, K. Yang, M. Yasunaga, D. Wang, Z. Li, J. Ma, I. Li, Q. Yao, S. Roman, et al. Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing*, pages 3911–3921, 2018.

X. Yu et al. AgentHER: Hindsight experience replay for LLM agents. 2025b. arXiv:2603.21357.

S. Yuan et al. iStar: Improving LLM agent reasoning via reinforcement learning with verifiable rewards. 2025. arXiv:2509.19199.

G. Zhang, H. Geng, X. Yu, Z. Yin, Z. Zhang, Z. Tan, H. Zhou, Z. Li, X. Xue, et al. The landscape of agentic reinforcement learning for LLMs: A survey. *Transactions on Machine Learning Research*, 2026.

R. Zhang et al. Agentic entropy-balanced policy optimization. 2025a. arXiv:2510.14545.

R. Zhang et al. Agentic reinforced policy optimization. 2025b. arXiv:2507.19849.

A. Zhao, Y. Wu, Y. Yue, T. Wu, Q. Xu, M. Lin, S. Wang, Q. Wu, Z. Zheng, and G. Huang. Absolute zero: Reinforced self-play reasoning with zero data. *arXiv preprint arXiv:2505.03335*, 2025.

Y. Zhong et al. Turn-level reward for multi-turn reinforcement learning. 2025. arXiv:2505.11821.

S. Zhou, F. F. Xu, H. Zhu, X. Zhou, R. Lo, A. Sridhar, X. Cheng, T. Ou, Y. Bisk, D. Fried, et al. WebArena: A realistic web environment for building autonomous agents. *arXiv preprint arXiv:2307.13854*, 2024.

Y. Zhou et al. SWEET-RL: Training multi-turn LLM agents on collaborative reasoning tasks. 2025. arXiv:2503.15478.

D. M. Ziegler, N. Stiennon, J. Wu, T. B. Brown, A. Radford, D. Amodei, P. Christiano, and G. Irving. Fine-tuning language models from human preferences. *arXiv preprint arXiv:1909.08593*, 2019.